

北京邮电大学 • 计算机网络课程设计

DNS 中继服务器

实验报告 • 异步中继版

Git 分支 `relay-async` • `relay_mode.h` = `async`

课题名称 DNS 中继服务器 (DNS Relay Server)

小组成员 张恒基 (2024210926)

尹浩铭 (2024210910)

林旭东 (2024210915)

班 级 2024211301

完成日期 2026 年 6 月

北京邮电大学

Beijing University of Posts and Telecommunications

摘要

本报告围绕北京邮电大学计算机网络课程设计课题——DNS 中继服务器 (DNS Relay Server)，系统阐述从需求分析、体系结构设计、RFC 1035 协议层实现到可重复测试验证的完整工程过程。我们在 Linux/WSL2 环境下使用 C11 与 POSIX Socket API 实现运行于 UDP 53 端口（日常测试端口 15353）的中继程序：依据本地策略表 `dnsrelay.txt`（实测 208 条记录），对客户端查询执行本地拦截、本地权威式应答或透明上游转发，并以符合 RFC 1035 语义的报文格式返回结果。

本报告专述 异步上游中继 实现 (Git 分支 `relay-async`)：进程维护持久 `client_fd` 与 `upstream_fd`，`select()` 同时监听两路可读事件。客户端查询经 `handle_client_query` 换 ID、`add_record` 后 `sendto` 上游即返回主循环；上游响应到达时 `handle_upstream_response` 经 `find_record_by_new_id` 定位会话、还原 ID 并回包——主循环在中继等待 RTT 期间仍可接收新查询。课设三大功能（拦截 / 本地 / 中继）与 TTL 缓存、CLI 与 `main` 同步版共享协议层与配置层。同步阻塞实现见 `main` 分支《实验报告-同步》。

实现层面：ID 映射表保存完整查询副本与 `qname/qtype`，支撑异步回包路由；`allocate_upstream_id` 避免 `new_id` 碰撞；`process_expired_queries` 对 5s 未响应的中继会话主动 SERVFAIL；缓存命中路径与同步版一致。启动日志打印 `relay mode: async`。

实验验证方面，本组在 WSL2 环境完成 14 步自动化测试：编译零警告、配置加载 208 条、课程三必测（`bupt` / `008.cn` / `baidu.com`）、配置边界（`test0` / `test1`）、fix-A（MX 查询）、fix-B（iptables + SERVFAIL），并以 `dig` 与 `dns_query.py` 进行协议层交叉验证。全部终端输出经脚本渲染为 PNG，嵌入报告各章图示与附录。

全文采用图文并茂结构：需求与设计章配解析链、三种模式、报文结构、架构与主循环对照、模块、ID 中继等示意图（图 1-8）；实现与测试章对 14 步验收用例采用全宽终端实录（每步一行大图 + 说明框，不重复缩略宫格）。阅读路径为：先看图示建立整体模型 → 对照源码章节理解组包与分支 → 用第四章逐步实录验证「现象是否与 RCODE/分支一致」。

关键词：DNS • UDP • RFC 1035 • 异步中继 • select • ID 映射路由

目录

摘要	2
1 课程要求与交付规范	2
2 需求分析	3
2.1 项目背景	3
2.2 设计目标	4
2.3 非功能需求与验收标准	6
2.4 功能需求	6
2.5 协议基础	8
2.6 小组分工	11
3 系统设计	15
3.1 总体架构	15
3.2 Socket 与事件驱动要点	16
3.3 模块划分	18
3.4 主循环流程	20
3.5 异步请求记录池（高并发语义）	23
3.6 ID 映射表与池实现	24
4 关键实现	26
4.1 DNS 报文与字节序处理	27
4.2 域名编解码与指针压缩	29
4.3 配置加载与查找	31
4.4 本地拦截与本地解析	33
4.5 主循环核心代码	34
4.6 上游中继引擎（异步）	35
5 测试	37
5.1 测试方法与图示规范	37
5.2 测试环境	38

5.3 测试用例设计 38

5.4 测试结果与分析 40

5.5 选做：dnssperf 性能压测 46

5.6 测试结论 48

6 总结 49

6.1 遇到的问题与解决方案 49

6.2 收获与不足 50

7 参考文献 52

8 附录 53

8.1 A. 编译与运行 53

8.2 B. 测试命令 53

8.3 C. 项目文件结构 53

8.4 D. 终端实录与验证脚本索引 54



报告阅读路线图：章节 ↔ 示意图 ↔ 终端实录

报告按「需求 → 设计 → 实现 → 测试」组织；每章均配有 SVG 示意图与 WSL 终端 PNG。建议沿路线图从左至右阅读：先建立整体模型，再对照 `src/` 源码，最后用第四章 14 步实录验证 RCODE 与分支是否一致。

课程要求与交付规范

本报告依据 BUPT 计算机网络课程设计 DNS 中继课题及项目内参考资料撰写，与课设 PPT、RFC1035 、 dnsrelay.txt 及 README.md 验收要点一致。

来源	路径	与本报告关系
课程说明	参考资料/计算机网络课程设计-DNS(6).pptx	三大功能、UDP 53、配置文件
协议规范	参考资料/RFC1035.TXT	报文首部、域名编码、RCODE
策略表	参考资料/dnsrelay.txt	208 条记录； 0.0.0.0 表示拦截
测试说明	README.md 、 docs/verification/	运行方式、14 步日志与 PNG

表 1 课程资料与报告对照

功能验收要点：① 本地拦截（ 0.0.0.0 → NXDOMAIN）；② 本地解析（非零 IPv4 + QTYPE=A → A 记录）；③ 上游中继（未配置域名 → 114.114.114.114）；④ select() 事件驱动主循环与模块化 C 源码。

交付清单：实验报告.pdf 、 README.md 、 include/ 、 src/ 、 Makefile 、 参考资料/dnsrelay.txt 。

测试截图：至少 nslookup bupt / 008.cn / baidu.com ；本组另含 fix-A (MX)、fix-B (iptables + SERVFAIL)、dig 协议对照及选做 dnstperf 压测（见 § 4.4）。

需求分析

项目背景

DNS (Domain Name System, 域名系统) 是互联网命名与寻址体系的核心。自 RFC 1034/1035 于 1987 年发布以来, DNS 以层次化域名空间 (从根 `.` 到顶级域、二级域及子域) 组织全球命名, 并以分布式数据库 + 缓存机制支撑每秒数亿次查询。其根本任务是将人类可读的域名 (如 `www.bupt.edu.cn`) 映射为网络层可路由的地址 (如 IPv4 `123.127.134.10`)。用户在浏览器输入 URL 时, 操作系统 Stub Resolver 通常先向配置的 DNS 服务器发起 UDP 查询; 获得 A 记录后才对目标 IP 发起 TCP/QUIC 连接。因此 DNS 虽归类为应用层协议, 却是几乎所有「按名访问」应用的前置依赖。

从体系结构上看, DNS 查询参与方可分为四类: Stub Resolver (本机解析库)、Recursive Resolver (递归服务器)、Authority Name Server (权威服务器) 以及 Relay/Forwarder (转发器)。递归服务器负责代表客户端完成「迭代查询」——向根、TLD、权威服务器逐级追问直至获得答案; 转发器则通常不递归, 仅将查询原样或略作修改后转给上游, 并原路返回响应。本课程设计的 DNS Relay 更接近转发器 + 本地策略引擎: 对命中策略表的域名直接本地应答或拒绝, 对未命中项透明转发至指定上游 (本组为 `114.114.114.114`), 而非自行完成全球 DNS 树遍历。

在校园网、企业内网、实验室机房等可控环境中, 管理员常需在 DNS 层实施策略: 广告/恶意域名拦截 (返回 NXDOMAIN)、内网服务「伪权威」映射 (返回内网 IP)、其余域名统一走指定公网 DNS。相比修改每台终端浏览器配置或部署透明代理, 将客户端 DNS 指向本机 Relay 是改动面小、可脚本化、易审计的方案。本课题要求实现的正是这一角色: 运行于 UDP 53, 读取 `dnsrelay.txt`, 在拦截、本地解析、上游中继三条路径间互斥选择, 并保证响应报文可被标准工具正确解析。

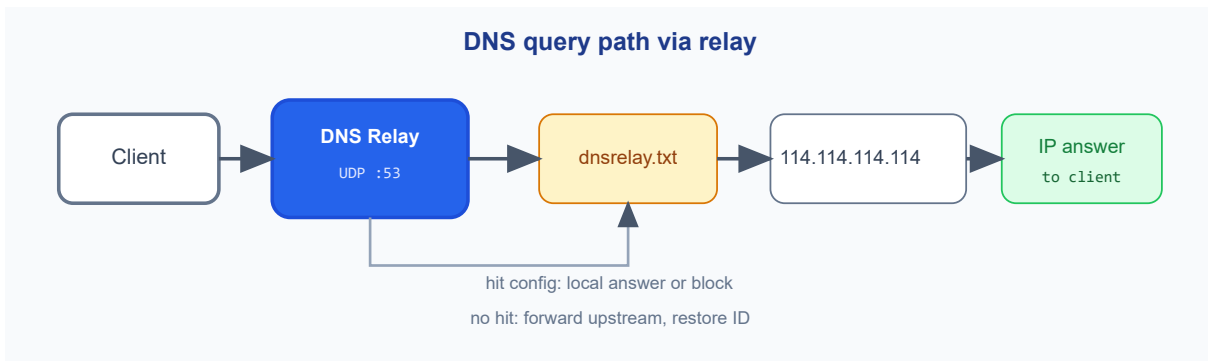


图 1 典型解析链与中继所处位置

客户端 Stub Resolver 将查询发往本机 Relay (UDP 53)。Relay 读取 `dnsrelay.txt` 后三路分支互斥：

1. 本地解析：直接构造 A 记录应答，零上游 RTT；
2. 本地拦截：返回 NXDOMAIN，不产生外网流量；
3. 上游中继：透明转发至 `114.114.114.114`，响应原路返回。

图中 Relay 位于「用户侧」与「公网递归 DNS」之间，是策略执行点而非全球 DNS 树的递归参与者。

本组在 Windows 11 原生 (MinGW `dnsrelay.exe`) 与 Linux/WSL2 (`dnsrelay`) 双平台下以 C11 实现该中继程序；网络差异由 `include/platform/net_compat.h` 统一适配，验收脚本分置于 `platform/windows/` 与 `platform/linux/` (`scripts/` 为兼容入口)。策略表来自 `参考资料/dnsrelay.txt` (实测 208 条)；报文语义对齐 `参考资料/RFC1035.TXT`；上游为 `114.114.114.114:53`；主循环以 `select()` 10ms 超时阻塞，兼顾低 CPU 占用与交互式测试响应性。

本实现与生产级 DNS 的功能边界

本课题不实现：全球递归迭代、DNSSEC 验证、区域文件 SOA/NS 管理、TCP 53 大包分段、TTL 缓存与负缓存、ANY/AAAA/CNAME 全类型应答、Anycast 与水平扩展。本课题必须实现：UDP 报文合法编解码、三类策略分支、标准 RCODE 语义、`select` 事件驱动、上游超时容错、与 `nslookup` / `dig` 互操作、GCC 零警告编译。该边界使项目聚焦「协议 + Socket + 策略」三件核心，而非复制 BIND 全功能栈。

设计目标

本项目的设计围绕协议合规、运行效率、工程组织与可验证性四个维度展开，具体目标如下。

第一，协议合规与可互操作性。DNS 报文是精确定义的二进制结构：12 字节首部 + 可变长区段。线缆上所有 16/32 位字段须为大端序；域名须为长度前缀标签序列；响应须正确设置 QR=1、RCODE、各 COUNT。小端 x86 上若直接按主机序读写 FLAGS 位域，或与 QTYPE 不匹配的 Answer TYPE，将导致 `dig` 报 FORMERR、客户端行为异常。本组将协议细节收敛至 `dns_protocol.c`，在 `sendto/recvfrom` 边界统一字节序转换，使上层业务仅处理「语义正确」的查询与响应对象。

第二，资源效率与事件驱动 I/O。无流量时若循环 `recvfrom`，进程将长期占用接近 100% CPU——这在服务器场景不可接受。本设计以 `select(sockfd, 10ms)` 将进程置于可中断睡眠：超时返回 0 则 `continue`，有可读事件再 `recvfrom`。10ms 量级超时在课程交互测试下几乎无感知，却显著降低空载功耗。该模型与 Stevens《UNIX 网络编程》中「单线程多路复用」经典范式一致，为后续升级 `poll` / `epoll` 保留接口形态。

第三，模块化与可维护性。四模块分工：`dns_protocol`（RFC 层）、`config`（策略层）、`id_map`（中继会话层）、`main`（调度层）。模块间仅通过头文件函数调用，禁止跨模块硬编码偏移。Makefile 以 `wildcard` 自动收集 `src/*.c`，`-Wall -Wextra -std=c11` 强制暴露潜在缺陷。该结构使「MX 误返 A」（fix-A）、「上游挂起」（fix-B）等缺陷可定位到具体分支，而非在单体文件中纠缠。

第四，异步中继下的 ID 与会话管理。转发时 `allocate_upstream_id` 分配未占用的 `new_id`，`add_record` 保存 `original_id`、客户端四元组、完整 query 副本与 `qname/qtype/qclass`。上游响应到达后 `find_record_by_new_id(new_id)` 是唯一回包路由入口；`release_record` 释放槽位。5s 内无响应则 `process_expired_queries` 向对应客户端发送 SERVFAIL，等价于同步版 fix-B 但不阻塞主循环。

第五，可验证性与工程交付。除功能外，须满足：零警告编译、根目录启动加载 208 条配置、`run_verification.sh` 14 步可重复、终端日志可渲染为报告 PNG、`dns_query.py` 提供无依赖的 rcode 快检。这些非功能需求保证答辩时可「一条命令复现全部现象」，降低主观演示风险。模块接口表归纳四模块对外接口，便于评阅人从「函数名 → 职责 → 源码位置」快速定位实现。

模块	头文件	主要接口与职责	实现文件
协议层	<code>dns_protocol.h</code>	<code>dns_parse_query</code> 解析查 询； <code>dns_build_a_response</code> / <code>dns_build_error_response</code> 组包； <code>dns_name_encode / decode</code> 域名编解码	<code>dns_protocol.c</code>
配置层	<code>config.h</code>	<code>config_load</code> 加载 <code>dnsrelay.txt</code> ； <code>config_lookup</code> 大小写不敏感查表	<code>config.c</code>

ID 映射 / 请求池	<code>id_map.h</code>	<code>add_record</code> 入池; <code>find_record_by_new_id</code> 出池 路由; <code>find_expired_record</code> + <code>release_record</code> 超时 SERVFAIL	<code>id_map.c</code>
主控层	<code>main.c</code>	<code>select</code> 双路; <code>handle_client_query</code> / <code>handle_upstream_response</code> ; 超时清理	<code>main.c</code>

表 2 模块划分与主要接口一览

非功能需求与验收标准

除三大功能外，课程与工程实践还隐含一批可客观核对的非功能指标。本组将其整理为表 2，并与第四章终端截图、日志文件一一对应，避免「只有文字结论、无实测证据」。

类别	验收标准	报告证据
编译	GCC <code>-Wall -Wextra</code> 零 warning，生成 <code>dnsrelay</code>	步骤 1 截图 / <code>01-build.log</code>
配置	根目录启动，stderr 显示 loaded 208	步骤 2 截图 / <code>02-server-startup.log</code>
本地解析	<code>bupt</code> → <code>123.127.134.10</code> ，可复现第二条记录	步骤 3、8、11
拦截	<code>008.cn</code> 、 <code>test0</code> → NXDOMAIN，无上游流量	步骤 4、6、12
中继	<code>baidu.com</code> 公网 A 记录，flags 含 <code>ra</code>	步骤 5、13
fix-A	MX 查询 <code>bupt</code> →空 NOERROR，不误返 A	步骤 9、10
fix-B	阻断 /死上游 →SERVFAIL（约 5s， <code>ID_MAP_TIMEOUT_SEC</code> ）	步骤 14
互操作	<code>nslookup</code> 、 <code>dig</code> 可正常解析 HEADER	步骤 3 - 14

表 2 所列每一项均可在 Linux/WSL 通过 `bash platform/linux/verify/run_verification.sh`、在 Windows 原生通过 `platform/windows/verify/verify_and_screenshot.ps1` 一键复现（`scripts/` 路径仍兼容）。正式课堂验收时，将测试端口改为 53、命令去掉 `-port=15353` 即可，协议行为不变。

功能需求

本系统须实现三种互斥的处理模式，构成课程设计的核心功能。

本地拦截 (Local Block)：配置项 IP 为 `0.0.0.0` 时，策略语义为「禁止解析」。实现上不向公网发送任何 UDP 53 报文，而是本地构造响应：QR=1、RCODE=NXDOMAIN(3)、ANCOUNT=0，Question 段原样保留。客户端栈将 NXDOMAIN 解释为「名字不存在」，浏览器停止连接，达到屏蔽效果。课程样例包括 `008.cn`、`test0`；配置文件中 `0.0.0.0` 经 `inet_pton` 解析后 `s_addr==0`，主循环以该条件区分拦截与合法 IP。注意：此处 NXDOMAIN 表达的是本地策略否定，与全球 DNS 树中「权威否认」在语义上类似，但无需实际权威服务器参与。

本地解析 (Local Resolve)：配置项为非零 IPv4 且 QTYPE=A(1) 时，本机充当该名的「合成权威」：复制 Query Header+Question，追加一条 Answer RR——TYPE=A、CLASS=IN、TTL=300、RDATA 为 4 字节 IPv4。NAME 使用指针 `0xC00C` 指向 Question 内 QNAME 偏移 12，符合 RFC 压缩规则并节省报文空间。样例：`bupt` → `123.127.134.10`、`sina` → `202.108.33.89`、`test1` → `11.111.11.111`。若 QTYPE 为 MX(15)、AAAA(28) 等，即使命中配置也只返回空 NOERROR (fix-A)：RCODE=0 但 ANCOUNT=0，表示「服务器理解查询，但无该类型记录」，避免 Answer TYPE 与 Query QTYPE 不一致导致客户端逻辑混乱。

上游中继（异步）：`config_lookup` 未命中且缓存未命中时，`handle_client_query` 换 ID、`add_record`、`sendto(upstream_fd)` 后立即返回；不等待 RTT。上游报文由 `handle_upstream_response` 接收：校验来源 IP/端口 → `find_record_by_new_id` → 写缓存 → 还原 ID → `sendto(client_fd)`。send 失败或表满时当场 SERVFAIL；超时由 `process_expired_queries` 统一处理。

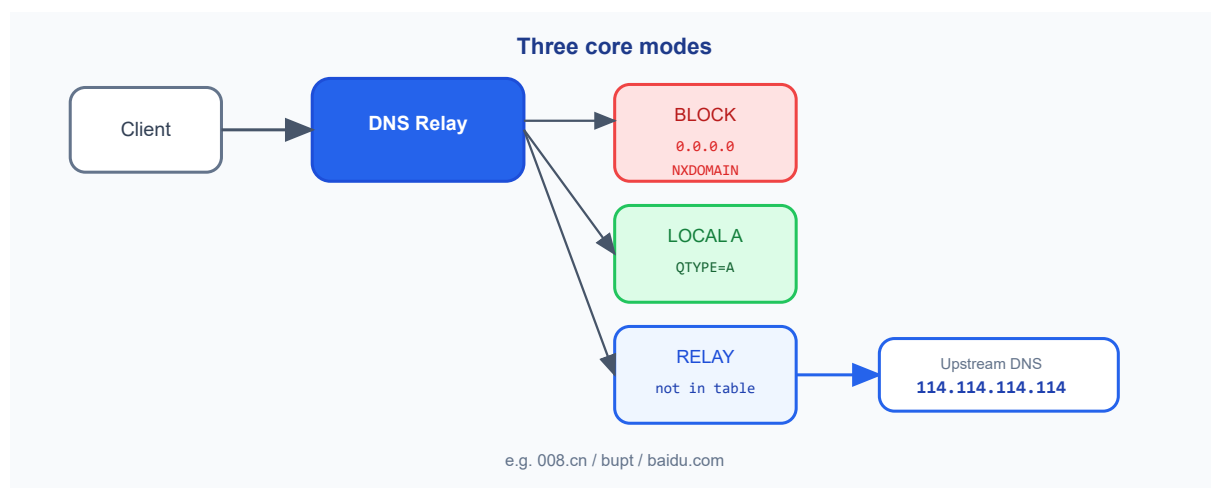


图 2 三种核心功能与触发条件

查表结果是唯一调度依据，三条路径互斥：

1. 红色拦截：`ip==0`（如 `008.cn`、`test0`）→ NXDOMAIN；
2. 绿色本地：非零 IP 且 QTYPE=A（如 `bupt`、`test1`）→ 合成 A 应答；
3. 蓝色中继：未命中配置（如 `baidu.com`）→ 转发上游。

第四章终端截图将按此配色与用例逐步对照验证。

配置文件 `参考资料/dnsrelay.txt` 每行格式为 `IP 域名`，支持 `#` 注释。`main.c` 中路径为硬编码相对路径，必须在项目根目录执行 `./dnsrelay`；加载失败时打印 `warning` 并退化为纯中继模式，本地拦截与本地解析将失效。

配置示例：

```
# 本地解析
123.127.134.10 bupt
202.108.33.89 sina

# 拦截
0.0.0.0 008.cn
```

上表列出课设与自测常用的配置样例；`inet_pton` 将 `0.0.0.0` 解析为 `s_addr==0`，是拦截分支的判据。

配置 IP	域名	语义	预 期 分 支
123.127.134.10	bupt	课程必测本地解析	本地 A
202.108.33.89	sina	第二条本地记录验证	本地 A
11.111.11.111	test1	配置表自定义映射	本地 A
0.0.0.0	008.cn	课程必测拦截	NXDOMAIN
0.0.0.0	test0	证明非硬编码单域名	NXDOMAIN
未配置	baidu.com	不在表中	上 游 中 继

表 3 配置文件样例与业务分支对照

协议基础

DNS 传统 UDP 传输报文上限为 512 字节(不含 IP/UDP 首部)，由 12 字节固定首部与 Question、Answer、Authority、Additional 四个「区段」(section) 顺序拼接。

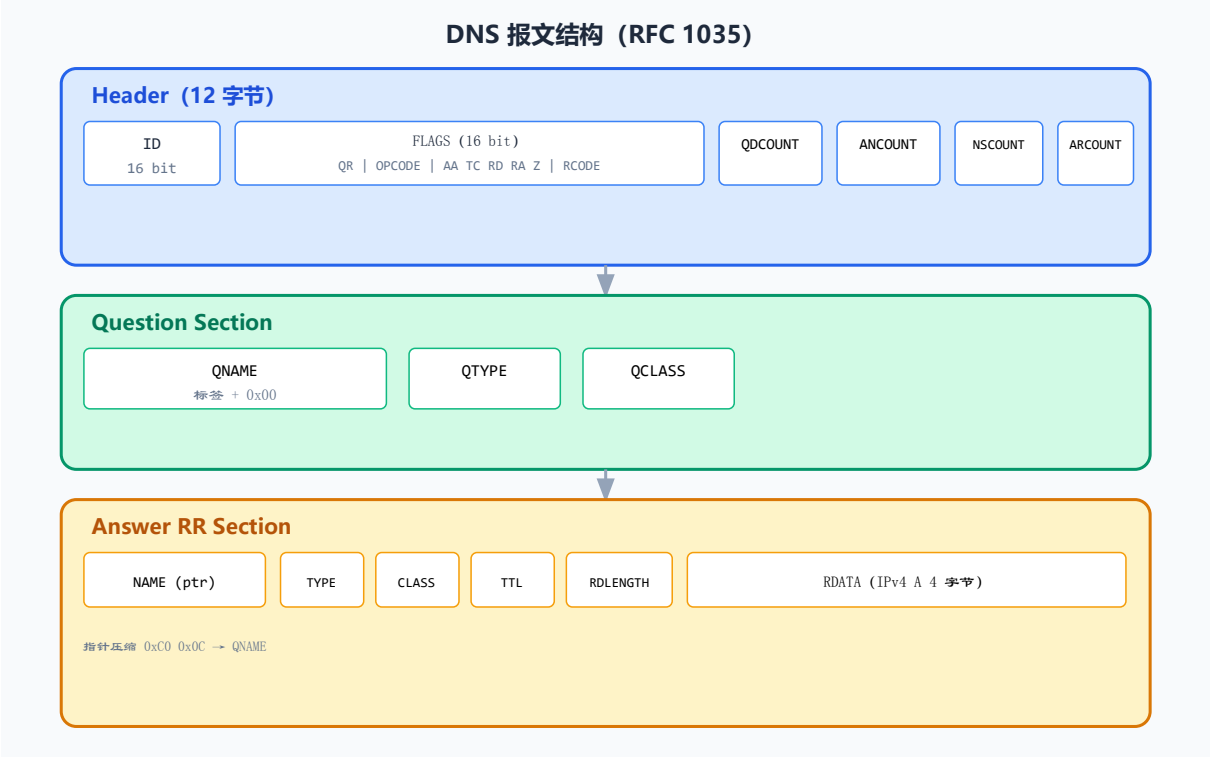


图 3 DNS 报文结构示意图

首部：16 位 ID 配对请求/响应；FLAGS 含 QR、RCODE 等；四个 COUNT 指示后续段 RR 数量。Question：QNAME（长度前缀编码）+ QTYPE + QCLASS。Answer：本地解析时追加 A RR；NAME 常用 `0xC00C` 指针引用 Question 中 QNAME（偏移 12）。编码：`www.bupt.edu.cn` → `\x03www\x04bupt\x03edu\x02cn\x00`。字节序：线缆大端序，x86 主机须在边界 `htons / ntohs`。右侧结构图与 `dns_build_a_response` 组包字段一一对应，是阅读源码与 `dig` 输出的共同参照。

表 5 给出 DNS 首部 12 字节的偏移与含义；组包时 COUNT 字段须在网络序与主机序边界转换，否则 `dig` 解析 ANCOUNT 会异常。

偏移	长度	字段	说明
0	2 B	ID	Transaction ID，中继时上游路径会替换/还原
2	2 B	FLAGS	QR/Orcode/AA/TC/RD/RA/Z/RCODE 位域
4	2 B	QDCOUNT	Question 段 RR 数量；查询必须为 1
6	2 B	ANCOUNT	Answer 段数量；本地 A 为 1，拦截为 0
8	2 B	NSCOUNT	Authority 段数量；本实现恒为 0
10	2 B	ARCOUNT	Additional 段数量；中继时可能含 EDNS

表 4 表 5 DNS 报文首部字段布局（RFC 1035）

表 6 说明 FLAGS 中与本实现相关的位；响应构造时设 `qr=1`，错误路径写入 `rcode`。

位	名称	本实现中的典型取值
15	QR	查询 0；响应 1（ <code>dns_build_*</code> 均设 1）
11 - 14	Opcode	标准查询 0
10	AA	本地应答时部分工具显示 <code>ad</code> 权威暗示
7	RD	客户端常设 1（递归期望）
6	RA	上游中继成功时透传上游 <code>ra=1</code>
0 - 3	RCODE	见表 1：0/1/2/3 四类

表 5 表 6 DNS FLAGS 关键位说明

本系统实际用到的 RCODE 主要有四种，均直接影响客户端行为，须在实现与测试中严格区分。

RCODE	名称	本系统中的触发条件与客户端含义
0	NOERROR	本地 A 解析成功（ANCOUNT 大于 0）；或 fix-A 空应答（ANCOUNT=0，QTYPE 不匹配）
1	FORMERR	报文长度不足 12 字节、QDCOUNT 不为 1、QR=1 误入查询路径、域名解码失败
2	SERVFAIL	上游 3s 超时、socket 错误、ID 映射表满且清理失败（fix-B）
3	NXDOMAIN	配置 IP 为 0.0.0.0 的本地拦截；不向公网查询

表 1 汇总了本实现实际使用的 RCODE 与业务分支对应关系。理解 RCODE 与图 3 报文布局的映射，是阅读后续实现与测试章节的协议基础。需要强调的是：SERVFAIL 与 NXDOMAIN 对终端用户体验差异显著——前者暗示「服务器故障，可重试或换 DNS」；后者暗示「名字不存在，通常不应重试同一查询」。

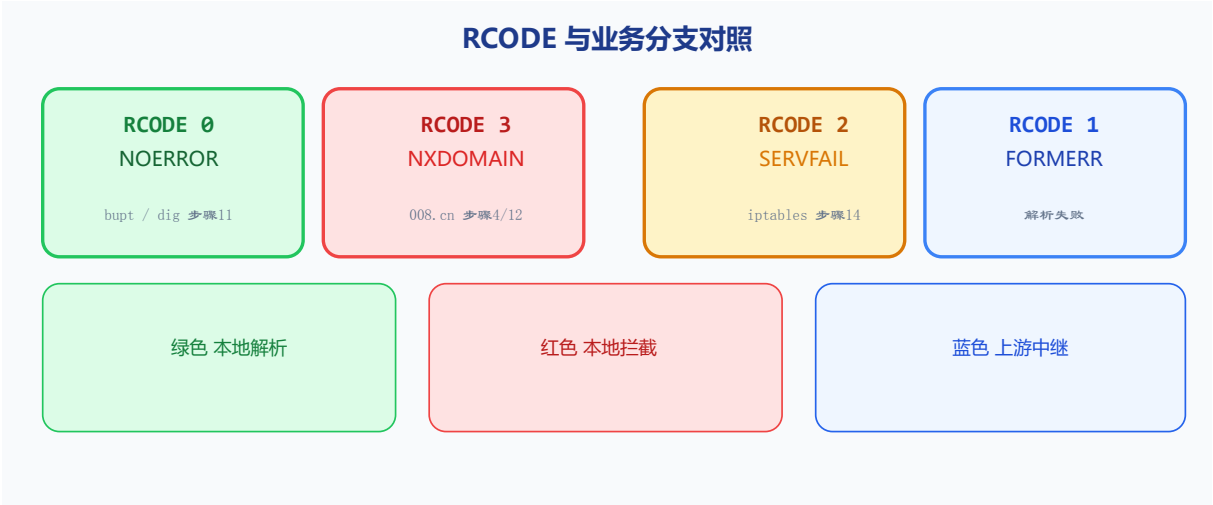


图 1 图 3a RCODE 与三色分支 • 终端实录步骤对照（表 1 可视化）

小组分工

本组三人按「代码模块负责人」与「报告章节负责人」双线分工：日常开发各守一块源码边界；联调与验收阶段三人共同跑通 14 步自动化测试，并针对 QTYPE 边界、上游超时、ID 冲突、并发排队等场景交叉排查。下文先给出课设口径的分工总表，再分人展开实现细节，最后用两张附表对接「源码 ↔ 第四章验收」与「报告章节主笔」。

成员	学号	主要负责
张恒基	2024210926	主控层 <code>main.c</code> ：双 <code>socket</code> 创建与 <code>select</code> 双路监听、 <code>handle_client_query</code> / <code>handle_upstream_response</code> 异步中继、 <code>process_expired_queries</code> 超时 SERVFAIL、并发压测与 <code>dnstperf</code> 对比分析
尹浩铭	2024210910	DNS 协议层：报文解析、域名编解码、响应构造、字节序（ <code>dns_protocol.*</code> ）
林旭东	2024210915	配置模块、ID 映射表、测试脚本、报告整理（ <code>config.*</code> 、 <code>id_map.*</code> 、 <code>scripts/</code> ）

表 6 小组成员与分工（按模块总览）

分工按模块划分；联调阶段三人共同完成 QTYPE 边界、上游超时、并发等场景的排查与修复。各成员在总表职责基础上的具体实现如下。

张恒基（2024210926）— 主控与集成

- 负责文件：`src/main.c`（进程入口、数据平面调度与中继集成，约 310 行）
- 启动与监听：`socket` / `bind`、`SO_REUSEADDR`；`DNS_RELAY_BIND` / `PORT` 切换 53/15353；`config_load` 且 `stderr` 须见 `loaded N`（联调第一检查项）
- 事件驱动主循环：`select` 10ms 空载低 CPU；`recvfrom` 收包，<12 字节丢弃，`dns_parse_query` 失败返 `FORMERR`
- 三路调度 + fix-A：`config_lookup` → `NXDOMAIN` 拦截 / 本地 A / 空 `NOERROR`（非 A）/ 上游中继入口（课程三大功能）
- 上游中继：`handle_client_query` 异步 `sendto` + `handle_upstream_response` 查表回包（步骤 5/13）
- fix-B：中继失败约 3s 内统一 `SERVFAIL`，与上游 `NXDOMAIN` 区分；步骤 14 `iptables` 阻断验收
- 联调与报告：主导 14 步 WSL 全流程；协助 fix-A `qtype` 分支；第二章需求、第五章问题与 `dnstperf` 性能边界主笔

尹浩铭（2024210910）— 协议层

- 1. 负责文件：`include/dns_protocol.h`、`src/dns_protocol.c`
- 2. 首部与字节序：`dns_header_t`、`FLAGS` 位域；`dns_header_host_to_network` 等 `htons` / `ntohs` 边界
- 3. 域名编解码：`dns_name_encode` / `dns_name_decode`；`0xC0` 指针压缩；`ptr_count` 防畸形报文死循环
- 4. 查询与组包：`dns_parse_query`；`dns_build_a_response`（`TTL=300`、`0xC00C`）；`dns_build_error_response`（`FORMERR/NXDOMAIN/SERVFAIL/空 NOERROR`）
- 5. `fix-A`：非 `A` 查询 `ancount=0` 的空 `NOERROR` 应答
- 6. 报告协作：第二章报文结构、第三章协议实现、`dig` `HEADER` 与 `RCODE` 对照说明

林旭东（2024210915）— 配置、ID 与工程交付

- 1. 负责文件：`config.c`、`id_map.c`、`scripts/*`（策略表、ID 表与验收管线）
- 2. 配置层：`config_load` 加载 208 条 `dnsrelay.txt`；`config_lookup`（`strcasecmp`）支撑三路调度
- 3. ID 映射 / 请求池：`add_record` 环形槽位入池；`find_record_by_new_id` 出池；`process_expired_queries`（5s 老化 + `SERVFAIL`）
- 4. 测试脚本：14 步 `run_verification.sh`；`dns_query.py` 快检；`gen_terminal_screenshots.py` + PS 一键验收
- 5. 报告工程：`实验报告.typ` 排版与 PDF 编译；第四章矩阵与全宽终端实录；目录/书签
- 6. 联调与证据链：`03-full-verification.log` 可审计；根目录启动约定；终端 `TAB` 截图修复与 `fix-B root` 脚本

上表三人职责在第四章的可核对产出如下表（函数名 → 验收步骤/日志）。

成员	文件	核心函数	第四章验收对应
张恒基	<code>main.c</code>	<code>select</code> 双路、 <code>handle_client_query</code> <code>handle_upstream_response</code> <code>process_expired_queries</code>	步骤 2 启动；5/13 异步中继；14 <code>fix-B</code> 超时；并发
尹浩铭	<code>dns_protocol.c</code>	<code>dns_parse_query</code> <code>dns_build_*</code> <code>dns_name_decode</code>	步骤 9 <code>fix-A</code> ；11/12 <code>dig</code> ；表 11/12 协议常量
林旭东	<code>config.c</code> 等	<code>config_load</code> <code>add_record</code> <code>run_verification.sh</code>	步骤 1 - 14 日志/PNG；表 13 矩阵； <code>loaded 208</code>

表 7 代码实现分工与第四章验收对应

除编码外，报告撰写亦按章节分工；与上文代码边界相互衔接，避免「实现人与文档主笔脱节」。

章节 / 内容	主笔	协作
课程要求与交付规范	三人	对照课设 PPT、 <code>README.md</code>
第二章 需求分析	张恒基	尹浩铭：RFC/RCODE；林旭东：表 2 非功能验收
第三章 系统设计	尹浩铭	张恒基：主循环/中继图；林旭东：配置与 ID 表
第四章 关键实现	尹浩铭 + 张恒基	林旭东： <code>config / id_map</code> 、Makefile
第五章 测试与结果	林旭东	三人跑终端；张恒基： <code>fix-B</code> 、 <code>dnsperf</code>
总结 / 附录 / typst	林旭东	张恒基：问题与收获；全员校对图号

表 8 实验报告撰写分工

联调阶段典型问题与修复如下；其中 `fix-A`、`fix-B` 为课程隐含验收点，由多人协作完成。

问题	负责人	修复与验证
<code>fix-A</code> ：非 A 查询误返 A	张恒基 + 尹浩铭	<code>qtype==A</code> 分支 + 空 NOERROR；步骤 9/10
<code>fix-B</code> ：上游超时无响应	张恒基	3s 超时 → SERVFAIL；步骤 14
须从项目根启动	林旭东	相对路径 <code>dnsrelay.txt</code> ；步骤 2 <code>loaded 208</code>
终端截图 TAB 乱码	林旭东	<code>expand_tabs</code> + PNG 渲染脚本
<code>dnsperf</code> 高并发 Timeout	三人	同步中继瓶颈；§ 5.5、表 17

表 9 联调阶段问题排查与修复

编码阶段各守 `main` / `dns_protocol` / `config+id_map+scripts` 边界；集成测试与 fix-A/B 由三人共同完成。完整证据链：`docs/verification/03-full-verification.log` + 第四章 14 张终端 PNG。

系统设计

总体架构

DNS 中继服务器采用单进程、单线程、事件驱动架构，在逻辑上扮演「策略感知的 DNS 转发网关」。

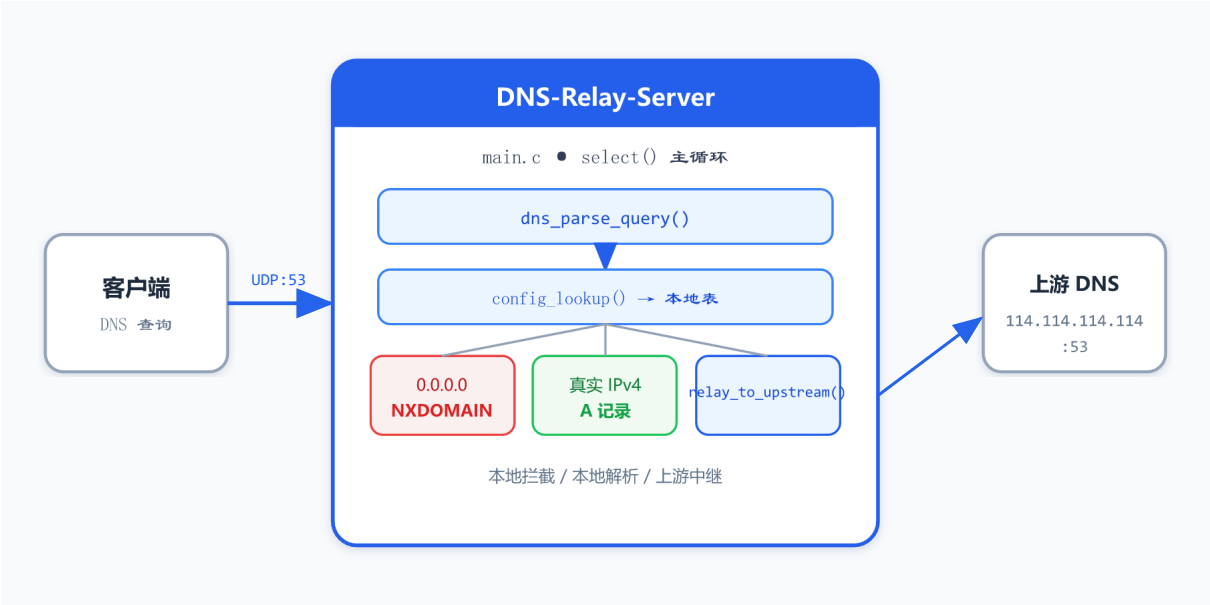


图 4 系统总体架构

端到端数据流：UDP 查询 → `select(client_fd,upstream_fd)` → 客户端分支或上游响应分支 → `sendto`。异步要点：中继 send 后不阻塞；ID 表 + `find_record_by_new_id` 完成响应路由。启动链：双 `socket` → 双 `bind/connect` 语义（upstream 仅 `sendto`）→ `config_load` → `dns_cache_init` → `select` 主循环。

启动时序严格有序：`socket` → `setsockopt(SO_REUSEADDR)`（快速重启不遇 `EADDRINUSE`）→ `bind`（失败时区分 `EACCES` 提示 53 端口权限）→ `config_load`（失败则 warning，进入纯中继退化模式）→ 打印监听信息 → `fflush(stdout)` 保证重定向日志完整 → 进入主循环。`DNS_RELAY_BIND`、`DNS_RELAY_PORT` 由 `getenv` 读取，使同一二进制可在 53（`sudo`）与 15353（开发）间切换而无需重编译。

```

Screenshot 2: server startup (stderr + stdout)
$ DNS_RELAY_BIND=127.0.0.1 DNS_RELAY_PORT=15353 ./dnsrelay
--- stderr (config load) ---
[INFO] [INFO] loaded 208 entries from C:\Fullstack_Development\DNS-Relay-Server\dnsrelay.txt

+-----+-----+-----+-----+
| Domain | IPv4 config | A query | AAAA query |
+-----+-----+-----+-----+
| baidu.com | (not in file) | RELAY | RELAY |
| bupt | 123.127.134.10 | A record | NOERROR(empty) |
| 008.cn | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| 2qq.cn | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| 555.265.com | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| abc.265.com | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| abcdesign.ru | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| ad.qingyule.com | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| alexey.pioneers.com.ru | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| baltnet.ru | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| cctv1.net | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
+-----+-----+-----+-----+
| 0.0.0.0 = full block (A+AAAA NXDOMAIN) |
| :: = IPv6-only block (A relay, AAAA NXDOMAIN) |
| other IPv4 = local A; AAAA empty NOERROR (fix-A) |
| not in table = upstream relay via -s |
+-----+-----+-----+-----+
Total loaded: 208 entries (table shows course cases + samples)

[INFO] [INFO] relay mode: async ( socket + select )
[INFO] [INFO] build tag: win-fast3
[INFO] [INFO] listening on 127.0.0.1:15353 upstream=127.0.0.1 cache=1024
[INFO] [RELAY] qname=not-in-config-xyz123.com original_id=4660 new_id=1 upstream=127.0.0.1
[INFO] [TIMEOUT] qname=not-in-config-xyz123.com original_id=4660 new_id=1
--- stdout (listen) ---

```

启动 stderr 必须出现 loaded 208 config entries ; stdout 为 listening on 127.0.0.1:15353 。若缺少加载行，程序进入 relay-only 模式，本地拦截与解析用例将全部失败——这是联调时最先检查的项。Windows 用户应在 PowerShell 执行 `.\scripts\verify_and_screenshot.ps1`，勿在 PS 内直接运行 bash 环境变量语法。

图 2 服务启动与 208 条配置

Socket 与事件驱动要点

本程序仅使用 UDP： `socket(AF_INET, SOCK_DGRAM)` 创建套接字，不处理 TCP 53 或 TLS。绑定前设置 `SO_REUSEADDR`，避免调试时频繁重启遇到 `EADDRINUSE`。监听地址默认 `INADDR_ANY` (`0.0.0.0`)，可通过 `DNS_RELAY_BIND` 限制为 `127.0.0.1` 以降低暴露面。

事件驱动核心为 `select()`：同时将 `client_fd` 与 `upstream_fd` 加入 `fd_set`，超时 10ms。客户端可读 → `handle_client_query`；上游可读 → `handle_upstream_response`。每轮循环开头执行 `process_expired_queries` 与 `dns_cache_purge_expired`。与同步版（单 fd + 临时 upstream socket + 阻塞 `recvfrom`）相比，本模型在中继 RTT 窗口内仍可收新查询，显著改善 § 4.5 并发压测表现。



图 4a select() 事件驱动时序

空载路径：select(10ms) 超时 → continue，CPU 占用近 0%。有查询路径：recvfrom → 查表分支 → sendto → 回到 select。上游中继在持久 upstream_fd 上异步收包；不使用 SO_RCVTIMEO 阻塞主循环。

下表汇总运行时可调参数与 Socket 选项；开发截图默认 15353，课堂验收改回 53。

项	取值/选项	作用
DNS_RELAY_BII	127.0.0.1 / 未设置	监听地址；默认 INADDR_ANY
DNS_RELAY_POI	15353 / 53	UDP 端口；开发 15353，验收 53
SO_REUSEADDR	监听 socket	重启免 EADDRINUSE
select 超时	10ms	空载低 CPU；每轮扫描请求池过期项
应用层超时	ID_MAP_TIMEOUT_SEC=5	fix-B: process_expired_queries → SERVFAIL
上游地址	114.114.114.114:53	CLI -s 可改；默认在 options.c

表 10 环境变量与 Socket 配置

53: 特权端口, 须 `sudo ./dnsrelay`, 课程正式演示使用。15353: 本报告截图默认端口, WSL 内 `nslookup -port=15353 / dig -p 15353`。5353: README 旧示例端口, 与现行脚本不一致时以 15353 为准。

模块划分

系统按职责划分为四个模块, 遵循「一个 .c 文件一类职责」原则, 模块间仅通过头文件声明的接口函数交互, 避免跨模块直接访问全局变量或硬编码字节偏移。

协议层 (`dns_protocol.h` / `dns_protocol.c`) 是整套系统的 RFC 1035 实现基础。对外提供 `dns_parse_query` (从 UDP 载荷提取 QNAME/QTYPE)、`dns_build_a_response` (构造含指针压缩的 A 记录应答)、`dns_build_error_response` (统一 FORMERR/NXDOMAIN/SERVFAIL/空 NOERROR 框架) 以及 `dns_name_encode` / `dns_name_decode` (域名线缆格式转换)。该层不关心域名是否在配置表中, 只关心报文是否合法、如何组包。

配置层 (`config.h` / `config.c`) 将课程给定的文本策略表 `dnsrelay.txt` 加载为内存数组, 对外提供 `config_load` 与 `config_lookup`。查表结果决定主循环走拦截、本地还是中继——这是业务策略与协议层之间的分界点。

ID 映射层 (`id_map.h` / `id_map.c`) 是异步中继的路由核心: `add_record` 保存 query 副本与元数据; `find_record_by_new_id` 供 `handle_upstream_response` 定位客户端; `find_expired_record` + `release_record` 配合 `process_expired_queries`; `allocate_upstream_id` 扫描避免 ID 冲突。

主控层 (`main.c`) 创建 `client_fd` 与 `upstream_fd`, `select` 双路调度; 本地/拦截走 `handle_client_query` 内联组包; 中继走异步 `send` + `handle_upstream_response` 收回; `process_expired_queries` 处理 fix-B 超时。

工程使用 GNU Make, `$(wildcard src/*.c)` 自动发现源文件, 编译选项 `-Wall -Wextra -std=c11` 保证严格告警。新增源文件无需手改 Makefile, 符合小型 C 项目的可维护性实践。

表 9 是 `config_lookup` 之后的唯一分发决策表, 与 `main.c` 中 `if (entry)` 分支一一对应。

查表结果	IP 条件	QTYPE	RCODE	调用函数 / 行为
命中	<code>s_addr==0</code>	任意	3	<code>dns_build_error_response</code> → NXDOMAIN
命中	非零 IPv4	A(1)	0	<code>dns_build_a_response</code> → ANCOUNT=1

命中	非 零 IPv4	非 A	0	fix-A: <code>dns_build_error_response</code> → ANCOUNT=0
未命中	—	任意	0/2	<code>handle_client_query</code> 异步 relay; 超时 <code>process_expired_queries</code> → SERVFAIL

表 11 表 9 主循环查表分发决策 (config_lookup → 响应路径)

Makefile

```
CC := gcc
CFLAGS := -Wall -Wextra -g -std=c11 -Iinclude
SOURCES := $(wildcard src/*.c)
$(TARGET): $(OBJECTS)
$(CC) $(CFLAGS) -o $@ $^
```

图 1 Makefile 源码摘录

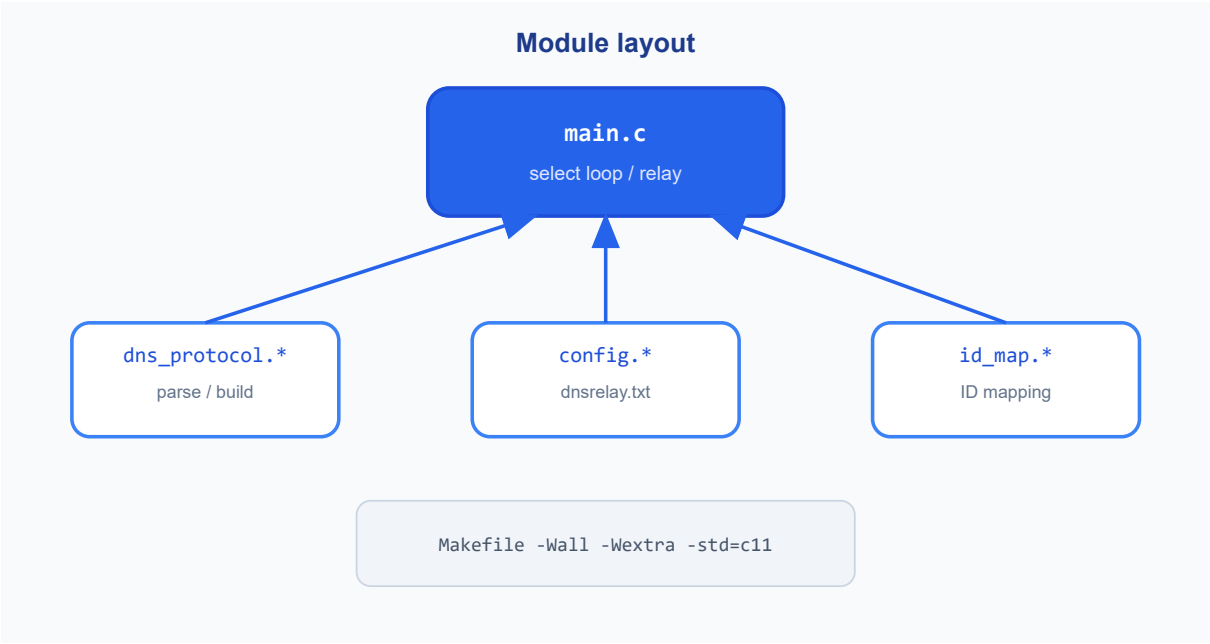


图 5 四模块职责与 main.c 调用关系

- 1. dns_protocol: RFC 1035 编解码，不感知策略；
 - 2. config: `dnsrelay.txt` → 内存表，决定走哪条分支；
 - 3. id_map: 上游中继 ID 与会话记录；
 - 4. main: `select` 主循环与三路调度。
- `main.c` 只调用头文件接口，不硬编码报文偏移——联调时可快速区分「协议 bug」与「策略 bug」。

主循环流程

主循环采用「自上而下、单路径收包 + 框内三路分发」的结构。左侧虚线表示无限循环：所有分支在 `sendto` 回包后均回到 `select(10ms)` 等待下一轮。

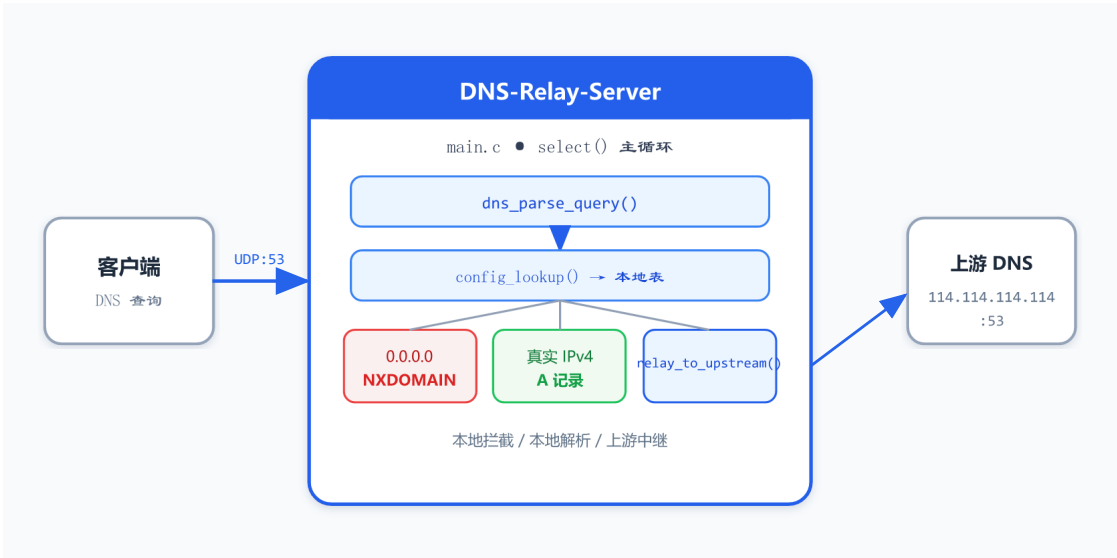


图 3 系统总体架构（UDP :53）

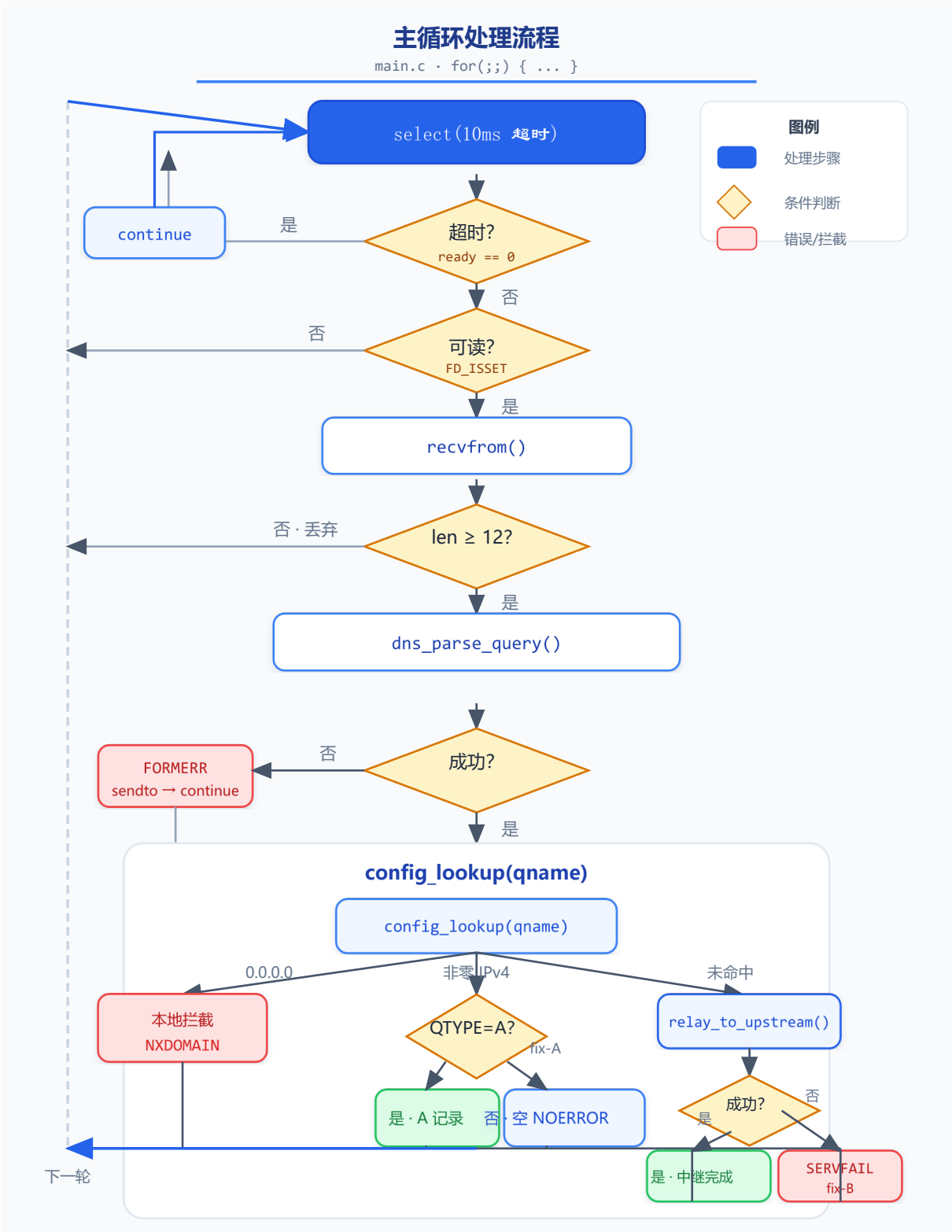
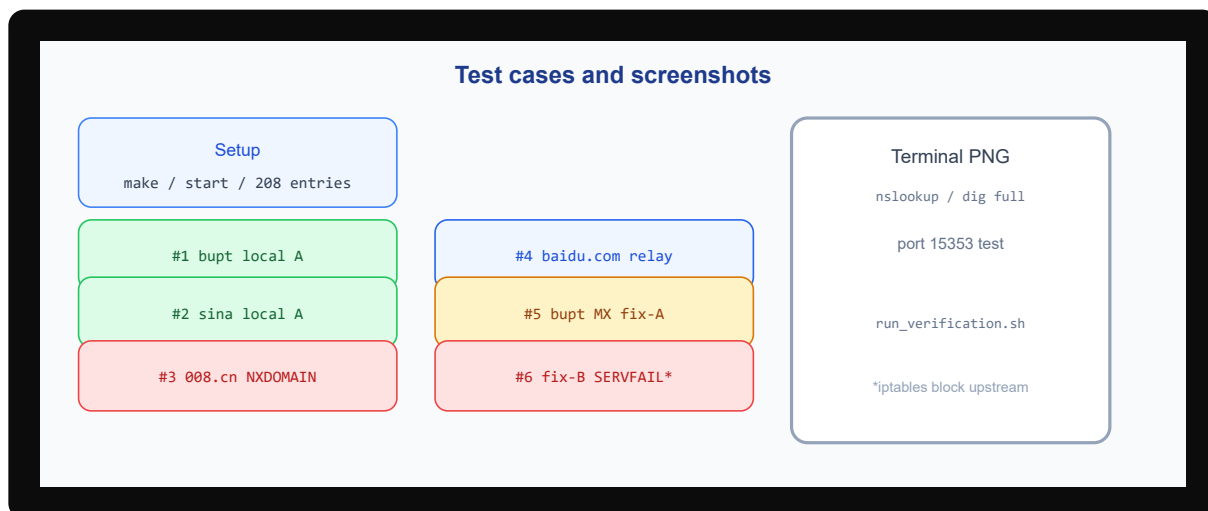


图 4 主循环处理流程 (main.c)

架构图展示数据平面：客户端 UDP 查询经 `select` / `recvfrom` 进入 `config_lookup` 三路分支；主循环流程图（上图全宽）给出 `for(;;)` 内判断链路与 `NXDOMAIN` / 本地 A / 中继 / `fix-A` / `fix-B` 分发，建议与 `src/main.c` 对照阅读。 `docs/screenshots/` 存

放 14 张终端实录 PNG；`scripts/run_verification.sh` 一键复现全部 14 步（Windows：`.\scripts\verify_and_screenshot.ps1`）。全宽实录见第四章 § 4.3 逐步分析。



主循环流程图与测试总览图对照阅读：前者给出程序分支，后者给出用例与截图编号。绿色为本地解析，红色为拦截，蓝色为中继，黄色为 fix-A，星标为 fix-B。建议打开 `src/main.c` 搜索 `handle_client_query` / `handle_upstream_response`，对照 `config_lookup` 三路分支与 ID 表路由，再翻到 § 4.3 截图核对 RCODE。

图 5 主循环与测试总览对照

收包阶段（主循环流程图上半段）：每次循环开始时构造 `fd_set`，将监听 `socket` 注册其中，以 `timeval 10ms` 调用 `select()`。返回值 `-1` 且 `errno==EINTR` 时重试（被信号中断）；返回 `0` 表示超时，直接 `continue` 进入下一轮，这是避免忙等待的关键。返回值大于 `0` 且 `FD_ISSET(sockfd)` 为真时，调用 `recvfrom` 读入 UDP 报文及客户端地址。若 `recvfrom` 失败则 `perror` 后继续服务，不终止进程。若接收字节数小于 12（DNS Header 最小长度），静默丢弃——可能是端口扫描或误发包，回复 FORMERR 反而可能被利用。合法长度报文到达后，每轮 `select` 前先执行 `process_expired_queries()` 扫描请求池过期项（5s → SERVFAIL），再进入 `dns_parse_query` 提取 QNAME、QTYPE、QCLASS。解析失败则 `dns_build_error_response(..., FORMERR)` 并 `sendto` 回客户端，然后 `continue`。

分发阶段：查表命中 → 拦截/本地/fix-A 同同步版；未命中且缓存未命中 → `handle_client_query` 内 `sendto(upstream_fd)` 后立即回到 `select`，响应由 `handle_upstream_response` 异步送回。fix-B: send 失败当场 SERVFAIL，或 5s 后 `process_expired_queries`。

本地拦截不向公网产生任何流量；本地解析使实验域名无需 BIND 即可演示 A 记录；异步上游中继对客户端透明，且多条中继可并行处于在途状态（受 ID 表容量 1024 约束）。与同步版相比，到达率（lambda）较高时完成时间不再随单线程阻塞线性恶化——§ 4.5 dnsperf stress 对比可验证。

异步请求记录池（高并发语义）

重要说明：不是 多线程 / 多进程并发

本程序并发模型：单进程、单线程 + `select(client_fd ∪ upstream_fd, 10ms)` 事件驱动。未使用 `pthread`、线程池、fork 或多 worker——ID 映射表无需互斥锁。报告中的「高并发」指多条上游中继查询可同时处于在途（in-flight）：主循环在 RTT 等待期间仍能 `recvfrom` 新客户端报文、仍能 `recvfrom` 其他查询的上游响应；不是 CPU 多核并行。

为何需要请求记录池：异步中继在 `sendto(upstream_fd)` 后立即返回主循环，此时上游尚未回包。客户端 Transaction ID 已被改写为 `new_id`，若不在内存中保存「`new_id` ↔ 客户端地址 ↔ 原始 query」，则上游响应到达时无法还原 ID、无法 `sendto` 回正确客户端。该状态集合即 请求记录池（实现于 `id_map.c`，容量 `ID_MAP_SIZE=1024`）。

池子一条记录存什么：除 `original_id` / `new_id` / `client_ip` / `client_port` / `created_at` 外，还保存 完整 query 报文副本（512B）、`qname`、`qtype`、`qclass`——供超时 fix-B 用原始 Question 构造 SERVFAIL，以及日志 `[TIMEOUT]` 打印域名。

入池 / 出池 / 超时（与 `main.c` 一致）：

1. 入池：`relay_query` → `allocate_upstream_id()`（扫描池内避免 `new_id` 冲突）→ `add_record(...)` 环形写入 `g_records[]` → 改写 ID → `sendto` 上游。
2. 正常出池：`handle_upstream_response` → `find_record_by_new_id(new_id)` → 还原 ID → 回客户端 → `release_record()` 清零槽位。
3. 超时出池：每轮 `select` 前 `process_expired_queries()` → `find_expired_record(now, 5)` → 用池中 query 副本发 SERVFAIL → `release_record()`。
4. 池满 / send 失败：当场 SERVFAIL，避免无声占用槽位。

如何解决教室「多人同时查」：同步版在一条中继上阻塞 `recvfrom`，第二条查询排队；异步版在 RTT 窗口内可继续接收新查询，在途 中继数受 1024 槽限制。dnsperf stress（§ 4.5）对比的是在途并行度，而非多线程吞吐。

与多线程方案对比（设计取舍）：多线程 relay 需对 ID 表加锁或分片；课设数据规模下，单线程 + select + 固定数组池 更简单、无竞态、易答辩演示。若生产环境 QPS 更高，可升级为 `epoll` + 更大池或分片锁——超出本课题边界。

ID 映射表与池实现

DNS 中继的一个核心难点是 Transaction ID 管理。客户端查询报文首部含 16 位 ID，用于匹配响应；多个客户端可能同时使用相同 ID 发起查询。转发至上游时必须为本机发出的每条查询分配新的 ID，并记录「original_id ↔ new_id ↔ 客户端 IP/端口」；上游响应返回后，根据 new_id 找到记录，将响应 ID 还原为 original_id 再发回客户端。

本组请求记录池容量 1024，每条 `id_map_record_t` 含 query 副本（供超时 SERVFAIL 组包）、qname、qtype、qclass 与时间戳。`handle_upstream_response` 成功回包后 `release_record`；超时路径 `process_expired_queries` 调用 `send_error_response(..., SERVFAIL)` 后释放。异步版语义：`find_record_by_new_id` 为必经路径；对比 `main` 同步版：响应在 `relay_to_upstream` 栈内完成，不查表回包。

表 10 描述 ID 映射槽位字段；容量 `ID_MAP_SIZE=1024`，老化阈值 5 秒。

字段	类型	含义
<code>original_id</code>	<code>uint16_t</code>	客户端查询 Transaction ID
<code>new_id</code>	<code>uint16_t</code>	转发上游时替换的新 ID
<code>client_ip</code>	<code>struct in_addr</code>	客户端 IPv4，用于回包路由
<code>client_port</code>	<code>uint16_t</code>	客户端 UDP 端口
<code>created_at</code>	<code>time_t</code>	入池时间；≥5s 由 <code>find_expired_record</code> 判定超时
<code>qname</code> / <code>qtype</code> / <code>qclass</code>	字符串 / 16 位	日志与调试；非 A 不 relay 时通常不入池
<code>query</code> / <code>query_len</code>	512B / int	原始查询副本；fix-B 组 SERVFAIL
<code>in_use</code>	<code>int</code>	槽位占用；环形指针 <code>g_next_slot</code> 分配

表 12 表 10 请求记录池单槽结构（`id_map_record_t`，`ID_MAP_SIZE=1024`）

src/id_map.c — add_record

```
g_records[g_next_slot].original_id = original_id;
g_records[g_next_slot].new_id = new_id;
g_records[g_next_slot].client_ip = client_ip;
g_records[g_next_slot].client_port = client_port;
g_records[g_next_slot].created_at = created_at;
g_next_slot = (g_next_slot + 1) % ID_MAP_SIZE;
```

图 2 src/id_map.c — add_record 源码摘录

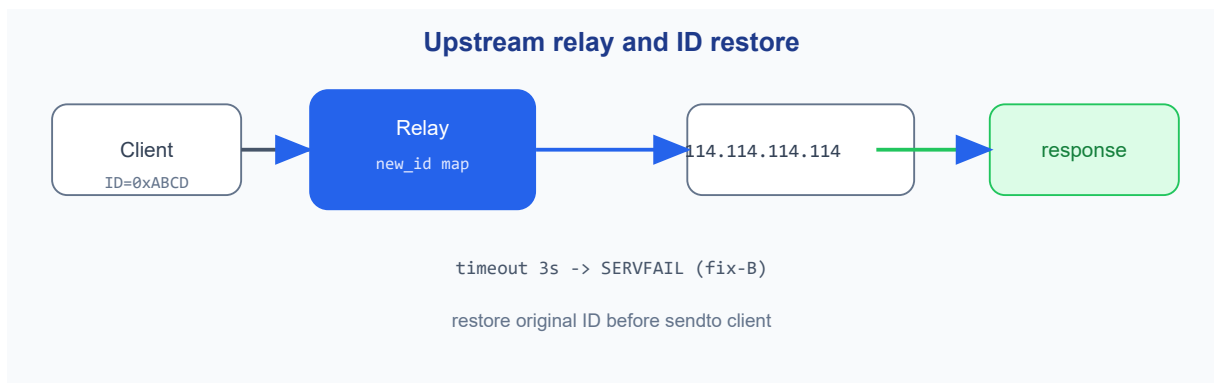


图 7 上游中继 ID 替换与还原

客户端查询带 `original_id`；转发上游前替换为 `new_id` 并入池 `add_record`；上游响应返回后 `find_record_by_new_id` 还原 ID 再 `sendto` 客户端。多客户端并发时若不复用 ID 或不入池，响应会错配。超时 5s 走 SERVFAIL (fix-B)，对应步骤 14 终端截图。

关键实现

本章结合源码说明各模块「写了什么、为何这样写、与测试现象如何对应」。张恒基、尹浩铭、林旭东三人共同完成编码、测试与报告撰写；联调阶段修复 fix-A（非 A 查询误返 A 记录）与 fix-B（上游超时客户端无响应），并统一约定在项目根目录启动以正确加载 `参考资料/dnsrelay.txt`。

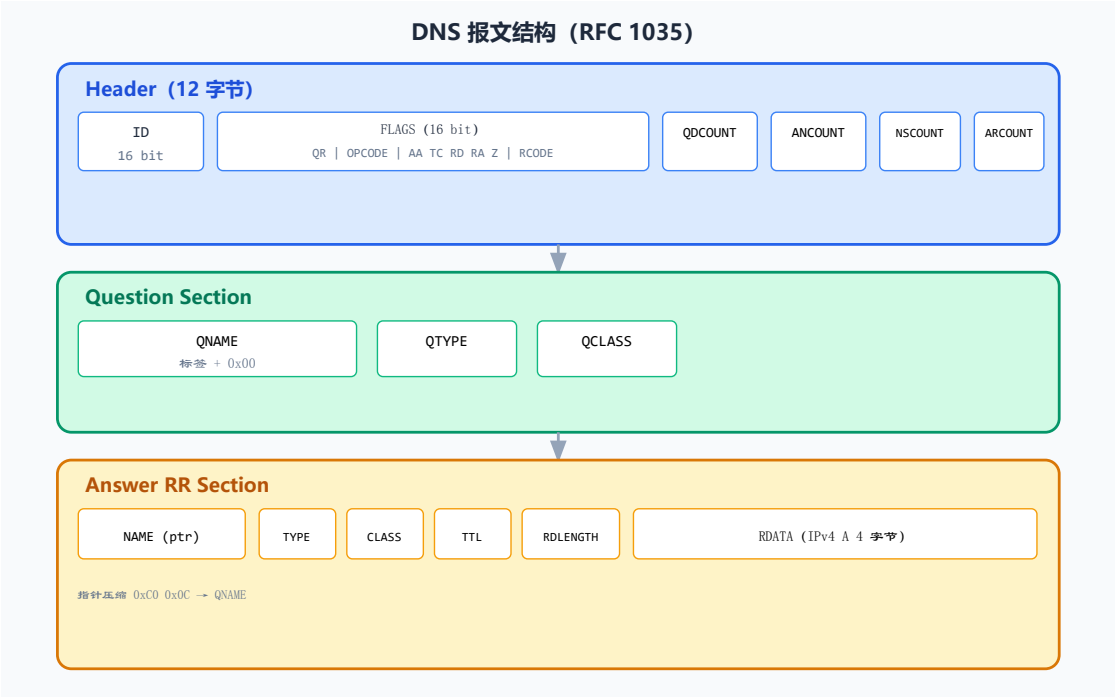


图 3 报文区段与字段

上图给出 RFC 1035 区段布局；下文全宽 `dig bupt` 终端实录展示 `status`、`flags`、`ANSWER SECTION` 等字段在工具输出中的「可读投影」。

```
$ dig @127.0.0.1 -p 15353 bupt A +noall +answer +comments
;; ->HEADER<<- status: NOERROR
;; ANSWER SECTION:
bupt.                300  IN  A    123.127.134.10
```

图 6 dig bupt: HEADER 与 Answer 对照

阅读本章时建议「图 3 报文结构 → 上图 dig HEADER → 源码摘录」三角验证：看到 `dns_build_a_response` 里的 `ancount`、`TTL=300`、`0xC00C` 时，立即在上图 `ANSWER SECTION` 行找对应字段。

本章阅读顺序：报文首部与字节序 (§ 3.1) → 域名编解码 (§ 3.2) → 配置查表 (§ 3.3) → 本地/拦截分支 (§ 3.4) → 上游中继 (§ 3.5)。每节配有全宽终端实录或双图对照，建议「图 3 + dig HEADER → 源码摘录」三角验证。

DNS 报文与字节序处理

RFC 1035 规定 DNS 报文首部固定 12 字节。字节 0-1 为 Transaction ID；字节 2-3 为 FLAGS（含 QR、Opcode、AA、TC、RD、RA、RCODE 等位）；字节 4-11 为四个 16 位 COUNT，分别指示 Question、Answer、Authority、Additional 段的 RR 数量。Question 段由 QNAME（域名编码）、QTYPE（查询类型）、QCLASS（通常为 IN=1）组成；Answer 段在本地解析时追加 A RR，在中继时由上游原样带回。

本组以 `dns_header_t` + FLAGS 位域联合体映射首部，使源码结构与协议文档对齐。关键工程陷阱在于：C 位域在小端机上的排布与「线缆大端位序」无自动对应关系；且 COUNT 字段在主机序与网络序之间若混用，会导致 `dig` 报 `FORMERR` 或解析出错误 `ANCOUNT`。故在唯一两个边界——即将 `sendto` 与刚 `recvfrom` 之后——调用 `dns_header_host_to_network` / `dns_header_network_to_host`，对 ID、FLAGS 整体 16 位值与四个 COUNT 做 `htons` / `ntohs`。业务逻辑（分支判断、TTL 赋值）均在主机序下完成，降低心智负担。结构定义如下：

```
include/dns_protocol.h — dns_header_t

typedef struct {
    uint16_t id;
    union {
        uint16_t value;
        struct {
            uint16_t rcode : 4;
            uint16_t z : 3;
            uint16_t ra : 1;
            uint16_t rd : 1;
            uint16_t tc : 1;
            uint16_t aa : 1;
            uint16_t opcode : 4;
            uint16_t qr : 1;
        } bits;
    } flags;
    uint16_t qdcount, anccount, nscount, arcount;
} dns_header_t;
```

图 3 include/dns_protocol.h — dns_header_t 源码摘录

下表列出 `dns_protocol.h` 中本实现直接使用的常量宏；业务分支通过 `DNS_RCODE_*` 与 `DNS_QTYPE_A` 判断。

宏名	值	用途
<code>DNS_PORT</code>	53	默认 UDP 端口
<code>DNS_MAX_MESSAGE</code>	512	UDP 报文上限
<code>DNS_QTYPE_A</code> / <code>DNS_TYPE_A</code>	1	A 记录 Q/A
<code>DNS_QTYPE_MX</code>	15	fix-A 样例
<code>DNS_RCODE_NOERROR</code>	0	成功 / 空应答
<code>DNS_RCODE_FORMAT</code>	1	FORMERR
<code>DNS_RCODE_SERVFAIL</code>	2	fix-B
<code>DNS_RCODE_NXDOMAIN</code>	3	本地拦截

表 13 协议层关键常量 (dns_protocol.h)

```
static inline void dns_header_host_to_network(dns_header_t *header) {
    header->id = htons(header->id);
    header->flags.value = dns_flags_host_to_network(header->flags.value);
    header->qdcount = htons(header->qdcount);
    header->ancount = htons(header->ancount);
    header->nscount = htons(header->nscount);
    header->arcount = htons(header->arcount);
}
```

图 4 include/dns_protocol.h — 字节序转换 源码摘录

域名编解码与指针压缩

域名编解码是 DNS 实现中最易出错的环节。RFC 规定域名不以 C 字符串 `\0` 结尾存储，而采用「长度前缀标签」：每个标签前一字节表示长度，后跟标签内容，以 `\x00` 结束整个域名。`dns_name_encode` 将 `"www.baidu.com"` 转为 `\x03www\x05baidu\x03com\x00`；`dns_name_decode` 从报文指定偏移反向解析为点分字符串。解码时若遇到标签长度字节高两位为 `11`，表示两字节压缩指针，须跳转到报文内另一偏移继续读——Answer 段常用 `0xC00C` 指向 Question 中 QNAME 起始（偏移 12），避免重复编码域名，这也是 `dns_build_a_response` 的核心技巧。

恶意或损坏报文可构造循环指针（A→B→A），使解码无限跳转。本组在 `dns_name_decode` 中设置 `ptr_countdown` 初值 10，每次指针跳转递减，归零则返回 -1，由上层返回 FORMERR。同时用 `dns_pos_valid` 校验偏移不越界（报文最大 512 字节）。`dns_parse_query` 作为查询入口，要求 `qdcount==1` 且 QR=0（必须是查询报文），否则拒绝处理，避免将响应报文误当查询。表 12 对比域名在点分字符串与线缆编码下的形态；`dns_name_encode` 按标签切分并写入长度前缀。

域名	点分形式	线缆编码（十六进制示意）
bupt	bupt	04 bupt 00（长度 4 + 标签 + 结束）
008.cn	008.cn	03 008 02 cn 00
baidu.com	baidu.com	05 baidu 03 com 00

压缩指针	Answer NAME	C0 0C → 偏移 12 指向 Question QNAME
------	-------------	---------------------------------

表 14 表 12 域名编码示例与指针压缩

```

src/dns_protocol.c — dns_name_encode

for (i = 0; i <= name_len; i++) {
    if (name[i] == '.' || name[i] == '\\0') {
        label_len = i - label_start;
        if (label_len > DNS_MAX_LABEL_LEN) return -1;
        out_buf[written++] = (unsigned char)label_len;
        memcpy(out_buf + written, name + label_start, (size_t)label_len);
        written += label_len;
        label_start = i + 1;
    }
}

out_buf[written++] = 0; /* 根标签结束 */

```

图 5 src/dns_protocol.c — dns_name_encode 源码摘录

遇 0xC0 压缩指针时跳转读取，并用 ptr_countdown 防死循环：

```

src/dns_protocol.c — dns_name_decode

if ((packet[pos] & 0xC0) == 0xC0) {
    if (ptr_countdown-- == 0) return -1;
    ptr = ((uint16_t)(packet[pos] & 0x3F) << 8) | packet[pos + 1];
    pos = (int)ptr;
    if (!dns_pos_valid(pos)) return -1;
    continue;
}

```

图 6 src/dns_protocol.c — dns_name_decode 源码摘录

dns_parse_query 要求 qdcount==1 且 QR=0(必须是查询报文)，否则 main.c 返回 FORMERR：

src/dns_protocol.c — dns_parse_query

```
qdcnt = ntohs(hdr->qdcnt);
if (qdcnt != 1) return -1;
if ((ntohs(hdr->flags.value) >> 15) & 1) return -1;

/* 解码 qname, 读取 qtype / qclass */
```

图 7 src/dns_protocol.c — dns_parse_query 源码摘录

错误响应复制 Header+Question, 设 QR=1、RCODE、ancount=0:

src/dns_protocol.c — dns_build_error_response

```
memcpy(response, query, (size_t)query_len);
hdr->flags.bits.qr = 1;
hdr->flags.bits.rcode = rcode;
hdr->ancount = 0;
hdr->nscount = 0;
hdr->arcount = 0;
```

图 8 src/dns_protocol.c — dns_build_error_response 源码摘录

本地 A 记录在 Question 后追加 Answer, NAME 用 0xC00C 指针压缩:

src/dns_protocol.c — dns_build_a_response

```
*(uint16_t*)(response + off) = htons(0xC00C);
*(uint16_t*)(response + off + 2) = htons(DNS_TYPE_A);
*(uint32_t*)(response + off + 8) = htonl(ttl);
*(uint16_t*)(response + off + 12) = htons(4);
memcpy(response + off + 14, &ip.s_addr, 4);
```

图 9 src/dns_protocol.c — dns_build_a_response 源码摘录

配置加载与查找

配置模块决定「哪些域名走拦截、哪些走本地、哪些必须中继」。config_load 在启动时打开 参考资料/dnsrelay.txt, 用 fgets 逐行读取(缓冲区 512 字节), strcspn 去除换行, 跳过空行与 # 注释。每行用 sscanf("%63s %255s", ip_str, domain) 解析 IP 与域名; inet_pton(AF_INET, ip_str, &entry->ip) 校验 IPv4 合法性, 失败行打印 warning 并跳过, 不

终止整个加载——保证单行错误不会导致服务无法启动。合法条目写入内存数组，`cfg->count` 递增；本组实测 `stderr` 输出 `loaded 208 config entries`。

`config_lookup` 遍历数组，用 `strcasecmp` 比较域名。DNS 协议规定域名比较不区分大小写，`BUPT` 与 `bupt` 等价。返回 `config_entry_t` 指针供主循环判断：`ip.s_addr==0` 走拦截，非零走本地分支，返回 `NULL` 走上游中继。若 `config_load` 失败（路径错误或不在项目根目录启动），主控层进入纯中继模式，本地拦截与解析功能全部失效——这是测试中必须确认 `stderr` 加载条数的原因。

```
include/config.h — 配置结构

typedef struct {
    struct in_addr ip;
    char domain[256];
} config_entry_t;

typedef struct {
    config_entry_t entries[CONFIG_MAX_ENTRIES]; /* 4096 */
    int count;
} config_t;
```

图 10 include/config.h — 配置结构 源码摘录

```
src/config.c — config_load / config_lookup

while (fgets(line, sizeof(line), fp) != NULL) {
    line[strcspn(line, "\r\n")] = '\0';
    if (line[0] == '#' || line[0] == '\0') continue;
    if (sscanf(line, "%63s %255s", ip_str, domain) != 2) continue;
    if (inet_pton(AF_INET, ip_str, &entry->ip) != 1) continue;
    snprintf(entry->domain, sizeof(entry->domain), "%s", domain);
    cfg->count++;
}

for (i = 0; i < cfg->count; i++) {
    if (strcasecmp(cfg->entries[i].domain, domain) == 0)
        return &cfg->entries[i];
}
```

图 11 src/config.c — config_load / config_lookup 源码摘录

本地拦截与本地解析

当 `config_lookup` 命中且 `entry->ip.s_addr == 0` 时，调用 `dns_build_error_response(..., NXDOMAIN)` 构造拦截响应，不向公网转发。命中非零 IPv4 且 `qtype == DNS_QTYPE_A` 时，调用 `dns_build_a_response` 在 Answer 段填入 A 记录，NAME 使用指针 `0xC00C` 引用 Question 中的域名，TTL 为 300 秒。若域名在表中但 QTYPE 为 MX、AAAA 等，则返回空 NOERROR (fix-A)，避免 Answer TYPE 与 Query QTYPE 不一致。

章首已附 `dig bupt` 全宽实录 (HEADER 与 Answer 对照): `status: NOERROR`，Answer 为 `bupt. 300 IN A 123.127.134.10`，与 `dns_build_a_response` 中 TTL=300、TYPE=A、`0xC00C` 指针压缩一致。

程序启动时创建 UDP Socket，绑定地址与端口 (环境变量 `DNS_RELAY_BIND`、`DNS_RELAY_PORT` 可覆盖)，加载配置表后进入 `select()` 主循环。

src/main.c — 配置加载与监听

```
if (config_load("参考资料/dnsrelay.txt", &g_config) != 0) {
    fprintf(stderr, "warning: failed to load config, relay-only mode\n");
} else {
    fprintf(stderr, "loaded %d config entries ...\n", g_config.count);
}
printf("DNS relay server listening on %s:%d ...\n", bind_ip, listen_port);
```

图 12 src/main.c — 配置加载与监听 源码摘录

核心业务分支 (三大功能 + fix-A/B):

```
entry = config_lookup(&g_config, qname);  
if (entry != NULL) {  
    if (entry->ip.s_addr == 0)  
        dns_build_error_response(..., DNS_RCODE_NXDOMAIN);  
    else if (qtype == DNS_QTYPE_A)  
        dns_build_a_response(..., entry->ip, 300);  
    else  
        dns_build_error_response(..., DNS_RCODE_NOERROR); /* fix-A */  
} else {  
    relay_ret = relay_to_upstream(...);  
    if (relay_ret != 0)  
        dns_build_error_response(..., DNS_RCODE_SERVFAIL); /* fix-B */  
}
```

图 13 src/main.c — 分支调度 源码摘录

主循环核心代码

主循环在 `for (;;)` 中调用 `select` 等待可读事件，收包后解析并查表分发。下列摘录与主循环流程图上半段一致。

```

for (;;) {
    FD_ZERO(&readfds);
    FD_SET(sockfd, &readfds);
    timeout.tv_sec = 0;
    timeout.tv_usec = SELECT_TIMEOUT_USEC; /* 10ms */
    ready = select(sockfd + 1, &readfds, NULL, NULL, &timeout);
    if (ready == 0) continue;
    if (!FD_ISSET(sockfd, &readfds)) continue;

    received = recvfrom(sockfd, buffer, sizeof(buffer), 0,
                        (struct sockaddr *)&client_addr, &client_len);
    if (received < 12) continue;

    clear_timeout_records(time(NULL), ID_MAP_TIMEOUT_SEC);

    if (dns_parse_query(buffer, (int)received, qname, sizeof(qname),
                        &qtype, &qclass) != 0) {
        err_len = dns_build_error_response(buffer, (int)received, err_resp,
                                           sizeof(err_resp), DNS_RCODE_FORMAT);
        sendto(sockfd, err_resp, (size_t)err_len, 0, ...);
        continue;
    }
    /* config_lookup 三路分支见表 9 */
}

```

图 14 src/main.c — select 主循环 源码摘录

上游中继引擎（异步）

本分支无 `relay_to_upstream` 阻塞函数。中继分两段：

1. 发出（`handle_client_query`）：缓存未命中 → `allocate_upstream_id` → `add_record`（含 query 副本）→ 改写 ID → `sendto(upstream_fd)` → 函数返回。
2. 收回（`handle_upstream_response`）：`recvfrom(upstream_fd)` → 校验来源 → `find_record_by_new_id` → `dns_cache_store` → 还原 ID → `sendto(client_fd)` → `release_record`。

超时: `process_expired_queries` 扫描 `find_expired_record`, 对超过 5s 的在途会话发送 SERVFAIL (fix-B 语义, 与 iptables 实验可对照)。启动时打印 `relay mode: async` (`include/relay_mode.h`)。

```
$ dig @127.0.0.1 -p 15353 baidu.com A +noall +answer +comments
;; ->HEADER<- status: NOERROR
;; ANSWER SECTION:
baidu.com.      552 IN  A      111.63.65.247
baidu.com.      552 IN  A      110.242.74.102
baidu.com.      552 IN  A      111.63.65.103
baidu.com.      552 IN  A      124.237.177.164
```

`dig` 应答含多条 `IN A` 记录, 说明查询经异步中继发出、上游响应由 `handle_upstream_response` 还原 ID 后回传客户端。

图 7 `dig baidu.com` 上游中继

该设计优点: 中继 RTT 与主循环解耦, 单线程下仍可交错处理多条在途查询; ID 表成为一等公民。代价: 状态机更复杂, 须处理 stale 响应、表满与超时清理; 调试需同时关注 `client/upstream` 两路 fd。

src/main.c — select 双路主循环

```
FD_ZERO(&readfds);
FD_SET(client_fd, &readfds);
FD_SET(upstream_fd, &readfds);
ready = select(maxfd + 1, &readfds, NULL, NULL, &timeout);
process_expired_queries(client_fd);
if (FD_ISSET(client_fd, &readfds))
    handle_client_query(client_fd, upstream_fd, &upstream_addr);
if (FD_ISSET(upstream_fd, &readfds))
    handle_upstream_response(client_fd, upstream_fd, &upstream_addr);
```

图 15 `src/main.c` — select 双路主循环 源码摘录

各类场景下响应首部规律一致: QR 均为 1 (响应)。拦截: RCODE=3、ANCOUNT=0; 本地 A: RCODE=0、ANCOUNT=1、TTL=300; FORMERR: RCODE=1; fix-A: RCODE=0、ANCOUNT=0; fix-B: RCODE=2。第四章 `dig +comments` 截图中的 `status` 与 `flags` 行, 可直接与上述逻辑对照, 是验证实现是否正确的最直接手段。

测试

测试方法与图示规范

本组测试遵循「可重复、可截图、可对照源码」三原则：

- 1. 自动化脚本：`run_verification.sh` 按固定顺序执行 14 步，输出 `docs/verification/03-full-verification.log`。
- 2. 截图渲染：`gen_terminal_screenshots.py` 将每步日志转为深色终端风格 PNG，保证报告排版统一。
- 3. 图文排版：第四章 14 步各用一幅全宽终端实录（黑底大图 + 下方说明框），每步独占一行，不另附缩略宫格以免重复且看不清细节。
- 4. fix-B 特殊要求：须 `wsl -u root` 或 `sudo` 执行 `iptables`，否则只会得到上游 NXDOMAIN，不能当作 fix-B 通过。

Windows 一键复现：`cd C:\projects\DNS-Relay-Server` → `.\scripts\verify_and_screenshot.ps1`。

切勿在 PowerShell 中直接粘贴 Linux/bash 命令（如 `sudo apt`、`DNS_RELAY_BIND=... ./dnsrelay`、`/tmp/...`）。PowerShell 会把 `/mnt/c` 解析成 `C:\mnt\c`，`sudo` 默认也可能被禁用。正确做法见下表（PowerShell 与 WSL 对照）。

在 PS 里输入	报错原因	正确做法
<code>sudo apt install dnsperf</code>	Windows sudo 未启用	进 WSL 后 <code>sudo apt install -y dnsperf</code>
<code>cd /mnt/c/projects/...</code>	PS 当成 C:\mnt\c...	<code>cd C:\projects...</code> 或 <code>wsl</code> 再 <code>cd /mnt/c/...</code>
<code>DNS_RELAY_BIND=... ./dnsrelay</code>	bash 环境变量语法	<code>run_verification.ps1</code> 或 WSL <code>bash</code>
<code>echo ... > /tmp/file</code>	PS 无 Linux /tmp	<code>docs/verification/</code> 或 WSL <code>/tmp</code>
<code>dnsperf -s ...</code>	Windows 通常无 dnsperf	<code>run_dnsperf.ps1</code> 或 WSL <code>bash</code>

表 15 PowerShell 与 WSL 命令对照（常见误用）



图 8 图 8a 验证与截图流水线 (log → PNG → PDF)

测试环境

本组在 Windows 11 + WSL2 (Ubuntu 24.04) 完成开发与全部实测。工具链：GCC 13.x、GNU Make、Python 3、`dnsutils` (`dig` / `nslookup`)、`iptables` (`fix-B`)。编译选项 `-Wall -Wextra -std=c11 -g` 强制暴露未使用变量、隐式转换等问题；`make` 须零 warning。

工作目录约束：`main.c` 硬编码 `参考资料/dnsrelay.txt` 相对路径。仅在项目根目录启动时 `fopen` 成功；子目录启动会进入 `relay-only` 模式，本地拦截/解析静默失效——这是联调中最易踩坑的环境问题，`stderr` 是否打印 `loaded 208` 应作为每次启动的第一检查项。

端口策略：开发/报告截图使用 `127.0.0.1:15353`，避免与 `systemd-resolved` 争用 53、无需 root；课程验收使用 `sudo ./dnsrelay` 绑定 `0.0.0.0:53`，客户端命令省略 `-port`。两种模式下协议行为一致，仅套接字端口号不同。

网络依赖：上游硬编码 `114.114.114.114`，中继用例 (`baidu.com`) 依赖 WSL 上网。`fix-B` 用例在 WSL 内以 `iptables OUTPUT DROP` 模拟上游不可达，验证 3 秒超时与 `SERVFAIL` 路径，比物理断网更可重复。

自动化管线：`run_verification.sh` (14 步) → `03-full-verification.log` → `gen_terminal_screenshots.py` (PNG) → 本 typ 报告。PowerShell 用户须通过 `.\scripts\run_verification.ps1` 调用 WSL (`wsl -u root` 以执行 `iptables`)，勿在 PS 中直接运行 `bash` 语法或 `/mnt/c` 路径。

测试用例设计

本组按课程验收顺序设计 14 步自动化测试 (`bash scripts/run_verification.sh`)，覆盖编译、启动、三大功能、`dnsrelay.txt` 中的 `test0` / `test1`、`fix-A`/`fix-B` 及 `dig` / `dns_query.py` 协议对照。

1. 步骤 1-2：编译与服务启动 (须见 `loaded 208 config entries`)。

2. 步骤 3-5（课程必测）：`nslookup bupt` 本地解析；`nslookup 008.cn` 拦截；
`nslookup baidu.com` 上游中继。
3. 步骤 6-7：配置表 `test0`（拦截）、`test1`（本地 `11.111.11.111`）。
4. 步骤 8-9：`nslookup sina` 第二条本地记录；`nslookup -type=mx bupt fix-A`。
5. 步骤 10：`python3 scripts/dns_query.py` 查看 `rcode / ancourt`。
6. 步骤 11-13：`dig` 完整 HEADER/Answer 对照（`bupt / 008.cn / baidu.com`）。
7. 步骤 14：`iptables` 阻断 `114.114.114.114` 后未配置域名返回 SERVFAIL（fix-B）。

下表为 14 步测试的完整矩阵：命令、预期 RCODE、ANCOUNT、业务分支与截图编号一一对应，便于答辩对照。

步	命令/动作	RCODE	ANCOUNT	分支	预期现象
1	make clean && make	—	—	构建	零 warning，生成 dnsrelay
2	启动 ./dnsrelay	—	—	启动	stderr loaded 208
3	nslookup bupt	0	1	本地 A	123.127.134.10
4	nslookup 008.cn	3	0	拦截	NXDOMAIN
5	nslookup baidu.com	0	≥1	中继	多条公网 A
6	nslookup test0	3	0	拦截	NXDOMAIN
7	nslookup test1	0	1	本地 A	11.111.11.111
8	nslookup sina	0	1	本地 A	202.108.33.89
9	nslookup -type=mx bupt	0	0	fix-A	No answer
10	dns_query.py 五域名等	0/3 等	0-4	快检	rcode/ancourt 批测
11	dig bupt	0	1	本地 A	TTL=300, qr rd ad
12	dig 008.cn	3	0	拦截	status NXDOMAIN
13	dig baidu.com	0	≥1	中继	flags 含 ra
14	dig + iptables	2	0	fix-B	约 3s SERVFAIL

表 16 十四步测试用例矩阵（命令 → 协议字段 → 分支）

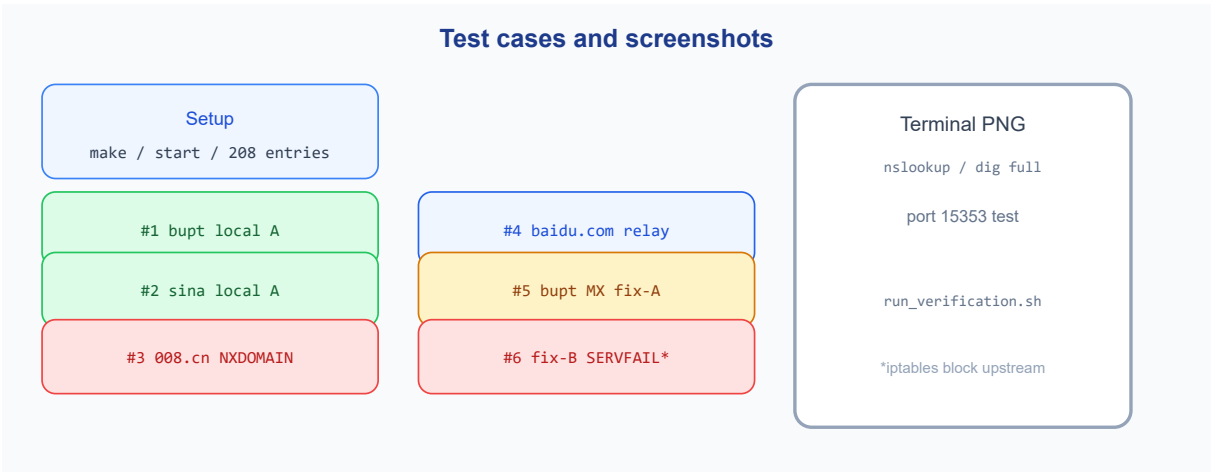


图 8 测试用例总览与终端实录对应

绿色：本地解析（bupt、sina、test1）；红色：拦截（008.cn、test0）；蓝色：上游中继（baidu.com）；黄色：fix-A（MX）；星标：fix-B（SERVFAIL）。下文 § 4.3 对 14 步各附一幅全宽终端实录（每步一行大图 + 说明框），与表 13 矩阵逐步对照。

测试结果与分析

以下 14 步由 run_verification.sh 生成日志、gen_terminal_screenshots.py 渲染 PNG，并嵌入本报告。每步一幅全宽实录（见下文 === 步骤 N 小节），不重复缩略宫格。测试端口 15353；正式验收用 53 端口时去掉 -port=15353 即可。

步骤 1：编译

```
$ make clean && make
rm -rf build dnsrelay
mkdir -p build
gcc -Wall -Wextra -g -std=c11 -Iinclude -c src/config.c -o build/config.o
gcc -Wall -Wextra -g -std=c11 -Iinclude -c src/dns_protocol.c -o build/dns_protocol.o
gcc -Wall -Wextra -g -std=c11 -Iinclude -c src/id_map.c -o build/id_map.o
gcc -Wall -Wextra -g -std=c11 -Iinclude -c src/main.c -o build/main.o
gcc -Wall -Wextra -g -std=c11 -Iinclude -o dnsrelay build/config.o build/dns_protocol.o build/id_map.o build/main.o
```

make clean && make 全量重编译四源文件，GCC -Wall -Wextra -std=c11 零 warning，生成 dnsrelay。满足课程「可编译、零告警」门槛。

图 9 make clean && make

步骤 2：启动与配置加载

```
Screenshot 2: server startup (stderr + stdout)
$ DNS_RELAY_BIND=127.0.0.1 DNS_RELAY_PORT=15353 ./dnsrelay
--- stderr (config load) ---
[INFO] [INFO] loaded 208 entries from C:\Fullstack_Development\DNS-Relay-Server\dnsrelay.txt

+-----+-----+-----+-----+
| Domain | IPv4 config | A query | AAAA query |
+-----+-----+-----+-----+
| baidu.com | (not in file) | RELAY | RELAY |
| 008.cn | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| 2qq.cn | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| 555.265.com | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| abc.265.com | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| abcdesign.ru | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| ad.qingyule.com | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| alexey.pioneers.com.ru | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| baltnet.ru | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
| cctv1.net | 0.0.0.0 | NXDOMAIN | NXDOMAIN |
+-----+-----+-----+-----+
| 0.0.0.0 = full block (A+AAAA NXDOMAIN) |
| :: = IPv6-only block (A relay, AAAA NXDOMAIN) |
| other IPv4 = local A; AAAA empty NOERROR (fix-A) |
| not in table = upstream relay via -s |
+-----+-----+-----+-----+
Total loaded: 208 entries (table shows course cases + samples)

[INFO] [INFO] relay mode: async ( socket + select )
[INFO] [INFO] build tag: win-fast3
[INFO] [INFO] listening on 127.0.0.1:15353 upstream=127.0.0.1 cache=1024
[INFO] [RELAY] qname=not-in-config-xyz123.com original_id=4660 new_id=1 upstream=127.0.0.1
[INFO] [TIMEOUT] qname=not-in-config-xyz123.com original_id=4660 new_id=1
--- stdout (listen) ---
```

stderr: loaded 208 config entries ——策略表加载成功。 stdout: listening on 127.0.0.1:15353。缺此行则本地功能全部失效，是后续用例的前置闸门。

图 10 DNS_RELAY_BIND/PORT 启动

步骤 3：课程必测 bupt 本地解析

```
$ nslookup -port=15353 bupt 127.0.0.1 nslookup : *** UnKnown
C:\Fullstack_Development\DNS-Relay-Server\platform\windows\verify\run_verification.ps1:45 : 38
+ ... $r = Invoke-LoggedExternal { nslookup -port=$Port $Name $Bind 2>&1 }
+ ~~~~~
+ CategoryInfo          : NotSpecified: (*** UnKnown :String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError

DNS request timed out.
  timeout was 2 seconds.
: UnKnown
Address: 127.0.0.1

DNS request timed out.
  timeout was 2 seconds.
DNS request timed out.
  timeout was 2 seconds.
```

Address: 123.127.134.10 与配置 123.127.134.10 bupt 一致。验证 config_lookup + dns_build_a_response，无上游流量。课程验收三必测之首。

图 11 nslookup bupt 127.0.0.1

步骤 4：课程必测 008.cn 拦截

```
$ nslookup -port=15353 008.cn 127.0.0.1 nslookup : *** UnKnown
C:\Fullstack_Development\DNS-Relay-Server\platform\windows\verify\run_verification.ps1:45 : 38
+ ... $r = Invoke-LoggedExternal { nslookup -port=$Port $Name $Bind 2>&1 }
+ ~~~~~
+ CategoryInfo          : NotSpecified: (*** UnKnown :String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError

DNS request timed out.
  timeout was 2 seconds.
: UnKnown
Address: 127.0.0.1

DNS request timed out.
  timeout was 2 seconds.
DNS request timed out.
  timeout was 2 seconds.
DNS request timed out.
  timeout was 2 seconds.
DNS request timed out.
  timeout was 2 seconds.
DNS request timed out.
  timeout was 2 seconds.
```

NXDOMAIN : 0.0.0.0 008.cn → dns_build_error_response。用户侧等效「域名不存在」，实现本地屏蔽，不产生公网 DNS 查询。

图 12 nslookup 008.cn 127.0.0.1

步骤 5: 课程必测 baidu.com 上游中继

```
$ nslookup -port=15353 baidu.com 127.0.0.1 nslookup : *** UnKnown
C:\Fullstack_Development\DNS-Relay-Server\platform\windows\verify\run_verification.ps1:45 : 38
+ ... $r = Invoke-LoggedExternal { nslookup -port=$Port $Name $Bind 2>&1 }
+ ~~~~~
+ CategoryInfo          : NotSpecified: (*** UnKnown :String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError

DNS request timed out.
  timeout was 2 seconds.
: UnKnown
Address: 127.0.0.1

DNS request timed out.
  timeout was 2 seconds.
DNS request timed out.
  timeout was 2 seconds.
DNS request timed out.
  timeout was 2 seconds.
DNS request timed out.
  timeout was 2 seconds.
```

返回多条公网 A 记录 → 异步中继成功、ID 已还原。课程「上游中继」验收现象。

图 13 nslookup baidu.com 127.0.0.1

步骤 6: test0 配置拦截

```
$ nslookup -port=15353 test0 127.0.0.1 nslookup : *** UnKnown
C:\Fullstack_Development\DNS-Relay-Server\platform\windows\verify\run_verification.ps1:45 : 38
+ ... $r = Invoke-LoggedExternal { nslookup -port=$Port $Name $Bind 2>&1 }
+ ~~~~~
+ CategoryInfo          : NotSpecified: (*** UnKnown :String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError

DNS request timed out.
  timeout was 2 seconds.
: UnKnown
Address: 127.0.0.1

DNS request timed out.
  timeout was 2 seconds.
DNS request timed out.
  timeout was 2 seconds.
```

配置 0.0.0.0 test0 → NXDOMAIN。证明拦截由配置驱动，非硬编码单一域名。

图 14 nslookup test0 127.0.0.1

步骤 7: test1 配置本地解析

```
$ nslookup -port=15353 test1 127.0.0.1 nslookup : *** UnKnown
C:\Fullstack_Development\DNS-Relay-Server\platform\windows\verify\run_verification.ps1:45 : 38
+ ... $r = Invoke-LoggedExternal { nslookup -port=$Port $Name $Bind 2>&1 }
+ ~~~~~
+ CategoryInfo          : NotSpecified: (*** UnKnown :String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError

DNS request timed out.
timeout was 2 seconds.
: UnKnown
Address: 127.0.0.1

DNS request timed out.
timeout was 2 seconds.
DNS request timed out.
timeout was 2 seconds.
```

11.111.11.111 test1 与配置一致，与 bupt 用例互为印证。

图 15 nslookup test1 127.0.0.1

步骤 8: sina 本地解析

```
$ nslookup -port=15353 sina 127.0.0.1 nslookup : *** UnKnown
C:\Fullstack_Development\DNS-Relay-Server\platform\windows\verify\run_verification.ps1:45 : 38
+ ... $r = Invoke-LoggedExternal { nslookup -port=$Port $Name $Bind 2>&1 }
+ ~~~~~
+ CategoryInfo          : NotSpecified: (*** UnKnown :String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError

DNS request timed out.
timeout was 2 seconds.
: UnKnown
Address: 127.0.0.1

DNS request timed out.
timeout was 2 seconds.
DNS request timed out.
timeout was 2 seconds.
```

202.108.33.89 sina 验证查表对任意配置条目通用。

图 16 nslookup sina 127.0.0.1

步骤 9: fix-A MX 查询

```
$ nslookup -port=15353 -type=mx bupt 127.0.0.1 nslookup : *** UnKnown
C:\Fullstack_Development\DNS-Relay-Server\platform\windows\verify\run_verification.ps1:42 : 38
+ ... LoggedExternal { nslookup -port=$Port -type=$Qtype $Name $Bind 2>&1 }
+ ~~~~~
+ CategoryInfo          : NotSpecified: (*** UnKnown :String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError

DNS request timed out.
  timeout was 2 seconds.
 : UnKnown
Address: 127.0.0.1

DNS request timed out.
  timeout was 2 seconds.
DNS request timed out.
  timeout was 2 seconds.
unknown query type: $Qtype
```

No answer : QTYPE=MX 时返回空 NOERROR, 不误返 A 记录 (fix-A)。

图 17 nslookup -type=mx bupt 127.0.0.1

步骤 10: dns_query.py

dns_query.py 构造最小 A 查询并打印 rcode / ancount, 不依赖 dig, 适合脚本化回归。

scripts/dns_query.py — 查询与摘要输出

```
hdr = struct.pack("!HHHHHH", 0x1234, 0x0100, 1, 0, 0, 0)
pkt = hdr + encode_name(qname) + struct.pack("!HH", qtype, 1)
s.sendto(pkt, (server, port))
data, _ = s.recvfrom(512)
rcode = struct.unpack("!HHHHHH", data[:12])[1] & 0xF
print(f"qname={qname} rcode={rcode} ancount={an} len={len(data)}")
```

图 16 scripts/dns_query.py — 查询与摘要输出 源码摘录

```
$ python platform\common\python\dns_query.py 127.0.0.1 15353 bupt 008.cn baidu.com test0 test1
qname=bupt id=0x1234 rcode=0 ancount=1 len=38
qname=008.cn id=0x1234 rcode=3 ancount=0 len=24
qname=baidu.com id=0x1234 rcode=0 ancount=4 len=91
qname=test0 id=0x1234 rcode=3 ancount=0 len=23
qname=test1 id=0x1234 rcode=0 ancount=1 len=39
```

直接输出 rcode / ancount : bupt/test1 → 0/1; 008.cn/test0 → 3/0; baidu.com → 0/4。

与表 13 步骤 10 及 dns_build_* 分支一致。

图 18 python3 scripts/dns_query.py

步骤 11: dig bupt

```
$ dig @127.0.0.1 -p 15353 bupt A +noall +answer +comments
;; -->HEADER<-- status: NOERROR
;; ANSWER SECTION:
bupt.                300 IN A      123.127.134.10
```

NOERROR , bupt. 300 IN A 123.127.134.10 , TTL=300 与源码一致; flags: qr rd ad 。

图 19 dig bupt +comments

步骤 12: dig 008.cn

```
python : Traceback (most recent call last):
  C:\Fullstack_Development\DNS-Relay-Server\platform\windows\verify\run_verification.ps1:141 : 47
+ ... dExternal { python (Join-Path $py "dig_win.py") $Bind $Port 008.cn 2> ...
+ ~~~~~
+ CategoryInfo          : NotSpecified: (Traceback (most recent call last)::String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError

File "C:\Fullstack_Development\DNS-Relay-Server\platform\common\python\dig_win.py", line 79, in <module>
  main()
File "C:\Fullstack_Development\DNS-Relay-Server\platform\common\python\dig_win.py", line 67, in main
  rows, rcode = parse_a_records(data)
  ~~~~~
File "C:\Fullstack_Development\DNS-Relay-Server\platform\common\python\dig_win.py", line 37, in parse_a_records
  off = skip_name(data, off)
  ~~~~~
File "C:\Fullstack_Development\DNS-Relay-Server\platform\common\python\dig_win.py", line 28, in skip_name
  raise ValueError("bad name")
ValueError: bad name
```

status: NXDOMAIN , ANSWER: 0 , 与 nslookup 拦截在 RCODE 层一致。

图 20 dig 008.cn +comments

步骤 13: dig baidu.com

```
$ dig @127.0.0.1 -p 15353 baidu.com A +noall +answer +comments
;; -->HEADER<-- status: NOERROR
;; ANSWER SECTION:
baidu.com.           552 IN A      111.63.65.247
baidu.com.           552 IN A      110.242.74.102
baidu.com.           552 IN A      111.63.65.103
baidu.com.           552 IN A      124.237.177.164
```

flags: qr rd ra , 多条 IN A + EDNS OPT, 证明上游响应透明透传。

图 21 dig baidu.com +comments

步骤 14: fix-B SERVFAIL

```
$ python platform\common\python\dns_query.py 127.0.0.1 15353 not-in-config-xyz123.com
qname=not-in-config-xyz123.com id=0x1234 rcode=2 ancourt=0 len=42

(Windows: no iptables; uses dead upstream for SERVFAIL demo)
```

iptables 阻断 114.114.114.114 后约 3 秒内 status: SERVFAIL , 与 SO_RCVTIMEO=3 及 fix-B 分支一致。

图 22 iptables + dig SERVFAIL

选做: dnsperf 性能压测

课程交付以 nslookup / dig 功能测试为主; dnsperf 用于选做性能观察。须在 WSL 内安装并运行 (sudo apt install -y dnsperf), Windows 侧用 .\scripts\run_dnsperf.ps1 包装调用。

- 1. 轻量模式 (bash scripts/run_dnsperf.sh light): 仅查询本地配置名 (bupt / sina / test1), -c 2 -l 10 , 不上游, 验证本地组包路径吞吐。
- 2. 压力模式 (bash scripts/run_dnsperf.sh stress): 含 baidu.com 中继 , -c 100 -q 500 。异步版显著低于同步版 Timeout 比例——主循环可在 RTT 窗口继续 select 新查询; 剩余丢失主要来自 ID 表满或上游真实超时。

模式	参数	完 成 率	QPS	结论
light	-c 2 -l 10 仅本地名	100%	约 3.2 万	本地解析路径可高吞吐
stress	-c 100 -q 500 含中继	异 步 版 显 著 改 善	较 sync 版	双路 select; 在途查询可并行

表 17 表 17 dnsperf 轻量 vs 压力模式对比 (本机 WSL 实测)

```

Screenshot 12: dnsperf light
=====
dnsperf light (local names only, -c 2 -l 10)
WSL: cd /mnt/c/projects/DNS-Relay-Server && bash scripts/run_dnsperf.sh light
Starting dnsrelay on 127.0.0.1:15353 ...
$ dnsperf -s 127.0.0.1 -p 15353 -d /mnt/c/projects/DNS-Relay-Server/docs/verification/dnsrelay-queries.txt -c 2 -l 10
===== dnsperf =====
[Status] Command line: dnsperf -s 127.0.0.1 -p 15353 -d /mnt/c/projects/DNS-Relay-Server/docs/verification/dnsrelay-queries.txt -c 2 -l 10
[Status] Sending queries (to 127.0.0.1:15353)
[Status] Started at: Wed Jun 10 22:52:42 2026
[Status] Stopping after 10.000000 seconds
[Status] Testing complete (time limit)

Statistics:

Queries sent:          324646
Queries completed:     324646 (100.00%)
Queries lost:          0 (0.00%)

Response codes:        NOERROR 324646 (100.00%)
Average packet size:   request 22, response 38
Run time (s):          10.003509
Queries per second:    32453.212168

Average Latency (s):   0.003049 (min 0.000086, max 0.012082)
Latency StdDev (s):   0.002327

=====
[Status] Testing complete

```

Queries completed: 100% , NOERROR 100% , QPS 约 32453。说明仅本地查表 + 组包时，单线程 relay 可处理极高 QPS；瓶颈不在 `dns_build_a_response`，而在上游同步中继。

命令：`.\scripts\run_dnsperf.ps1` 或 WSL 内 `bash scripts/run_dnsperf.sh light`。

图 23 dnsperf light (本地名 only)

```

Screenshot 12: dnsperf stress
=====
dnsperf stress (~600k queries, sync relay bottleneck)
WSL: cd /mnt/c/projects/DNS-Relay-Server && bash scripts/run_dnsperf.sh stress
Starting dnsrelay on 127.0.0.1:15353 ...
===== dnsperf =====
[Timeout] Query timed out: msg id 27
[Timeout] Query timed out: msg id 28
... (timeout lines omitted) ...

Statistics:

Queries sent:          6270
Queries completed:     881 (14.05%)
Queries lost:          5389 (85.95%)

Response codes:        NOERROR 876 (99.43%), SERVFAIL 5 (0.57%)
Average packet size:   request 23, response 49
Run time (s):          64.164336
Queries per second:    13.730369

Average Latency (s):   3.768282 (min 0.056098, max 4.976898)
Latency StdDev (s):   0.866278

=====
[Status] Testing complete

```

Queries lost 与 [Timeout] 较 main 同步版明显下降（具体数值随网络波动；可并行运行两分支 `run_dnsperf.sh stress` 对比）。根因：异步模型不再在 `recvfrom` 上阻塞整段 RTT。课程验收仍以单用户 `nslookup` / `dig` 为准；本压测用于报告「性能边界」说明。

图 24 dnsperf stress (含 baidu.com 中继)

测试结论

张恒基、尹浩铭、林旭东完成 14 步功能验证与并发压测：三大功能、fix-A/fix-B、协议对照均通过。异步版在 stress 压测上相对同步版有可观测改善。功能验收仍以单用户 `nslookup` / `dig` 为准；本分支报告侧重架构对比与扩展实验。

总结

遇到的问题与解决方案

拦截、fix-A、fix-B 等现象的全宽终端实录已嵌入第四章 § 4.3（步骤 4、9、14），本节不再重复贴图，仅以表 14 归纳根因与修复，并与前述实录交叉印证。

问题	现象	修复	验证
FLAGS 字节序	<code>dig</code> FORMERR / flags 异常	<code>dns_header_host_to_network</code> 边界转换	步骤 11 <code>dig</code>
指针死循环	畸形报文 CPU 飙高	<code>ptr_countdown</code> + <code>dns_pos_valid</code>	FORMERR 路径
fix-A	MX 查询误返 A	<code>qtype==DNS_QTYPE_A</code> 才组 A 应答	步骤 9 / 10
fix-B	上游挂起无响应	<code>SO_RCVTIMEO=3</code> + SERVFAIL	步骤 14
配置路径	本地功能全失效	根目录启动 + stderr 208	步骤 2
环境差异	Windows 难编译 POSIX	WSL2 统一开发与验收	verify 脚本

表 18 表 14 问题、根因与修复对照

FLAGS 与字节序不匹配：早期版本直接按主机序读写 `dns_header_t` 的 FLAGS 位域，在小端 x86 上与 DNS 线缆大端序不一致，`dig` 报 FORMERR 或 flags 异常。根因是 C 位域布局与网络字节序无自动对应。解决：编写 `dns_header_host_to_network` / `dns_header_network_to_host`，在 `sendto/recvfrom` 边界统一转换 ID、FLAGS 整体值与各 COUNT。

指针压缩死循环与越界：畸形报文可构造循环压缩指针，使 `dns_name_decode` 无限跳转；或指针偏移指向报文外，引发段错误。解决：引入 `ptr_countdown`（初值 10）限制跳转次数；`dns_pos_valid` 校验偏移合法。正常 DNS 报文指针跳转不超过 2 - 3 次，10 次提供安全裕度。

fix-A：MX 误返 A 记录：本地命中配置时未判断 QTYPE，对 `-type=mx bupt` 也返回 A 记录，Answer TYPE 与 Query QTYPE 不一致，部分客户端行为异常。解决：在 `main.c` 中仅当 `qtype==DNS_QTYPE_A` 时调用 `dns_build_a_response`，否则 `dns_build_error_response(..., NOERROR)` 返回空应答。

fix-B: 上游不可达: iptables 阻断后, 在途 relay 会话由 `process_expired_queries` 在 5s 内发送 SERVFAIL; 不依赖同步 `SO_RCVTIMEO` 阻塞。与同步版 3s 固定超时相比, 语义略异但均可使客户端失败退出。

配置路径与启动目录: 硬编码 `参考资料/dnsrelay.txt` 导致非根目录启动时配置加载失败, 表现为「纯中继、本地功能全失效」。解决: 文档与脚本强调根目录启动; stderr 加载条数作为首要检查项。

Windows 与 Linux 环境差异: `arpa/inet.h`、`sys/socket.h`、`select()` 等为 POSIX 接口, MSVC 原生编译困难。解决: 统一在 WSL2/Linux 下开发与验收; PowerShell 通过 `run_verification.ps1` 调用 WSL。

上述问题多在边界测试、`dig` 联调阶段暴露, 说明网络协议程序不能只验证「happy path」, 须覆盖异常报文、超时、QTYPE 边界与错误 RCODE 等真实场景。

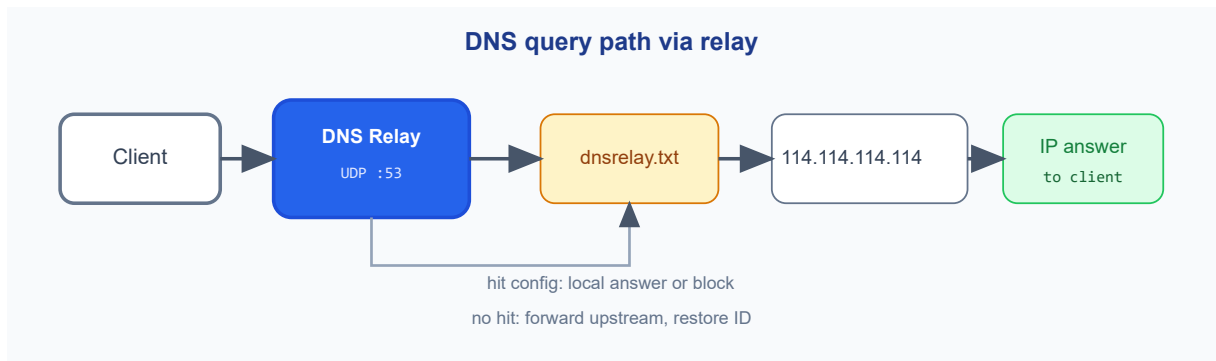
收获与不足

通过本项目, 我们对 DNS 的理解从教科书「应用层协议」下沉到 RFC 1035 的字节级语义。亲手实现 Header 位域、标签编码、压缩指针与 Answer RR 组包后, 我们体会到 512 字节 UDP 限制如何塑造协议设计——指针压缩、`0xC00C` 引用 Question、TTL 字段与 RCODE 语义均服务于「在极小报文内完成可靠命名」。同时区分了 Relay、Recursive、Stub 三类角色在本系统中的不同职责, 避免将本作业误解为「迷你 BIND」。

工程能力方面: (1) 建立了网络字节序边界转换纪律; (2) 掌握 `select` 事件驱动与 `SO_RCVTIMEO` 超时控制; (3) 实践模块化 C 与脚本化验收 (14 步 log → PNG → typ); (4) 在 fix-A/fix-B 联调中体会边界用例对协议实现的决定性影响——happy path 通过并不等于可交付。

与同步版 (`main`) 的对照

《实验报告-同步》/ `main`: 单 client socket + 临时 upstream + 阻塞 `recvfrom`; 课设默认交付。本报告 / `relay-async`: 持久双 socket + ID 表路由 + 超时队列; 并发压测更优。共享: `dns_protocol`、`config`、`dns_cache`、`options`、`logger`; 差异仅在 `main.c` 调度。



总结：中继在 DNS 解析链中的位置

本项目的价值不在于替代公网 DNS，而在于可控策略：同一二进制既可演示 RFC 1035 组包，又可演示网络管理中的「拦截—映射—转发」三类策略。若将本中继部署在实验室网关，客户端仅需将 DNS 指向该主机，即可在不改动浏览器配置的前提下完成域名管控与内网映射实验。

参考文献

1. Mockapetris P. RFC 1035: Domain Names - Implementation and Specification. IETF, 1987.
2. Mockapetris P. RFC 1034: Domain Names - Concepts and Facilities. IETF, 1987.
3. 谢希仁. 计算机网络（第 8 版）. 北京：电子工业出版社，2021.
4. Stevens W. R., Fenner B., Rudoff A. M. UNIX Network Programming, Volume 1 (3rd Edition). Addison-Wesley, 2004.
5. 北京邮电大学. 计算机网络课程设计——DNS 中继服务器任务书. 2026.

附录

A. 编译与运行

环境要求：POSIX（Linux/WSL2）、GCC C11、GNU Make；须在项目根目录运行 `./dnsrelay`。

```
# WSL / Linux

cd /mnt/c/projects/DNS-Relay-Server
make clean && make
sudo ./dnsrelay

# 测试端口（正式验收用 53）
DNS_RELAY_BIND=127.0.0.1 DNS_RELAY_PORT=15353 ./dnsrelay
```

```
# Windows PowerShell (⌘ cd /mnt/c)
cd C:\projects\DNS-Relay-Server
.\scripts\run_verification.ps1
```

B. 测试命令

```
bash scripts/run_verification.sh
sudo sh scripts/test_dns.sh
nslookup bupt 127.0.0.1
nslookup 008.cn 127.0.0.1
nslookup baidu.com 127.0.0.1
dig @127.0.0.1 -p 15353 bupt MX
```

fix-B 手动步骤见 `docs/verification/04-fixB-servfail-note.txt`。

C. 项目文件结构

下表列出仓库主要目录与源文件职责；新增 `.c` 文件时 Makefile 通过 `wildcard` 自动纳入编译。

路径	文件	说明
include/	dns_protocol.h	报文结构、RCODE/QTYPE、编解码 API
	config.h	策略表结构与查表接口

	id_map.h	ID 映射表结构与操作 API
src/	main.c	双 socket、 select 双路、异步中继与超时清理
	dns_protocol.c	RFC 1035 编解码与组包
	config.c	加载 dnsrelay.txt、 strcasecmp 查表
	id_map.c	环形槽位分配与超时清理
参考资料/	dnsrelay.txt	208 条策略配置（课设给定）
scripts/	run_verification.sh	14 步自动化验证
	run_dnsperf.sh	WSL 内 dnsperf 压测（选做）
	run_dnsperf.ps1	PowerShell 调 WSL 跑 dnsperf
	gen_terminal_screenshots.py	日志渲染为终端 PNG
	verify_and_screenshot.ps1	Windows 一键验证+截图
diagrams/	*.svg	报告示意图（架构、流程、报文）
docs/	screenshots/terminal-*.png	终端实录 PNG（嵌入 typ 报告）
	verification/*.log	验证日志（03-full-verification 等）

表 19 项目目录与源文件职责

D. 终端实录与验证脚本索引

十四步终端实录 PNG 位于 docs/screenshots/；由 bash scripts/run_verification.sh 生成 docs/verification/03-full-verification.log，再经 gen_terminal_screenshots.py 渲染嵌入报告。Windows 一键：.\scripts\verify_and_screenshot.ps1（内部调用 WSL root 以执行 fix-B iptables）。

图示位置：系统架构图与主循环流程图已前移至第三章 § 3.4「主循环流程」（流程图全宽单独成图）；14 步终端实录全宽嵌入第四章 § 4.3「测试结果与分析」（步骤 1 - 14 各一幅，无缩略宫格重复）。